

# Deploying a React + API + PostgreSQL application on AWS

## Approach A: Amazon EC2 · Approach B: Amazon ECS on AWS Fargate

A practical, diagram-first guide. Every configuration file is complete and uses the same names in both approaches. Complete files are in the companion **Appendix** PDF; this guide shows the relevant excerpt at each step.

| Scope | In scope / out of scope                                                                                                                                                                                                                   |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| In    | AWS infrastructure, Terraform (HCL), GitHub Actions (YAML), Dockerfile, ECS task definition, EC2 user-data, systemd, IAM JSON, AWS CLI, networking, security, scaling, deployment and rollback.                                           |
| Out   | Application internals (Spring Boot, React source code, business logic). The backend is treated as a black box that listens on port 8080, exposes <code>/actuator/health</code> , and reads <code>POSTGRES_*</code> environment variables. |

### Naming conventions used everywhere

| Thing                          | Name (placeholder values marked REPLACE in files)                                                                                                                                         |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Project / environment / region | <code>ai-lab</code> / <code>prod</code> / <code>us-east-1</code> → prefix <code>ai-lab-prod</code>                                                                                        |
| Domains                        | <code>app.example.com</code> (React via CloudFront) · <code>api.example.com</code> (API via ALB)                                                                                          |
| Network                        | VPC <code>ai-lab-prod-vpc</code> 10.20.0.0/16 · SGs <code>ai-lab-prod-alb-sg</code> , <code>-app-sg</code> , <code>-db-sg</code>                                                          |
| Load balancer                  | <code>ai-lab-prod-alb</code> · target group <code>ai-lab-prod-api-tg</code> · port 8080 · <code>/actuator/health</code>                                                                   |
| Database                       | RDS <code>ai-lab-prod-db</code> · database <code>ai_lab</code> · master user <code>ai_lab_admin</code> · RDS-managed secret <code>rds!db-...</code>                                       |
| App secret                     | Secrets Manager <code>ai-lab/prod/api</code> (JSON, e.g. <code>OPENAI_API_KEY</code> )                                                                                                    |
| EC2 only                       | ASG <code>ai-lab-prod-api-asg</code> · bucket <code>ai-lab-prod-artifacts-&lt;account&gt;</code> · SSM <code>/ai-lab/prod/api/release</code> · logs <code>/ec2/ai-lab-prod-api</code>     |
| ECS only                       | cluster <code>ai-lab-prod</code> · service + task family <code>ai-lab-prod-api</code> · container <code>api</code> · ECR <code>ai-lab/api</code> · logs <code>/ecs/ai-lab-prod-api</code> |
| Shared                         | frontend bucket <code>ai-lab-prod-frontend-&lt;account&gt;</code> · CI role <code>ai-lab-prod-github-deploy</code> · state bucket <code>ai-lab-tfstate-&lt;account&gt;</code>             |
| Placeholders                   | account <code>111122223333</code> · OWNER/ <code>ai-engineering-lab</code> · <code>example.com</code> · release/image tag = git SHA (e.g. <code>0123abc</code> )                          |

**What was checked** Terraform 1.16.5 + AWS provider 6.68.0: `terraform validate` passed for both stacks, and a full `terraform plan` ran green under `terraform test` with **mocked** AWS providers. Shell scripts passed `shellcheck`; workflows passed `actionlint`; JSON files parse. **Not done:** no `apply` against a real account, no Docker build, no load test. Treat the files as a reviewed starting point, not as proven-in-production configuration.

# Contents

|                                                                 |           |
|-----------------------------------------------------------------|-----------|
| <b>Part 1 · Architecture overview</b>                           | <b>4</b>  |
| 1.1 Side by side                                                | 4         |
| 1.2 Approach A: EC2                                             | 4         |
| 1.3 Approach B: ECS on Fargate                                  | 5         |
| 1.4 Required vs optional components                             | 6         |
| 1.5 EC2 vs ECS vs Fargate                                       | 7         |
| <b>Part 2 · Inventory of every configuration file</b>           | <b>8</b>  |
| 2.1 Directory trees                                             | 8         |
| 2.2 Approach A file inventory                                   | 8         |
| 2.3 Approach B file inventory                                   | 9         |
| 2.4 Where does a setting belong?                                | 10        |
| <b>Part 3 · EC2 deployment walkthrough</b>                      | <b>12</b> |
| 3.1 Deploying an application update                             | 16        |
| 3.2 Rolling back a failed release                               | 17        |
| <b>Part 4 · ECS Fargate deployment walkthrough</b>              | <b>18</b> |
| 4.1 The ECS vocabulary in one picture                           | 18        |
| 4.2 How tasks receive configuration and secrets                 | 22        |
| 4.3 Deploying new image versions                                | 23        |
| 4.4 Rolling deployments and automatic rollback                  | 23        |
| 4.5 Shell access and DB port forwarding without hosts           | 24        |
| <b>Part 5 · YAML, JSON, HCL and shell: file-by-file</b>         | <b>25</b> |
| 5.1 Who reads what                                              | 25        |
| 5.2 Execution order                                             | 25        |
| 5.3 Terraform HCL                                               | 25        |
| 5.4 GitHub Actions YAML                                         | 27        |
| 5.5 Dockerfile, ECS task definition JSON, IAM JSON              | 27        |
| 5.6 Shell, systemd, CloudWatch agent                            | 28        |
| <b>Part 6 · React hosting: S3 + CloudFront</b>                  | <b>29</b> |
| <b>Part 7 · Networking and security</b>                         | <b>31</b> |
| 7.1 VPC, subnets and routing                                    | 31        |
| 7.2 Security groups: permitted traffic paths                    | 31        |
| 7.3 Outbound access, NAT and VPC endpoints                      | 32        |
| 7.4 IAM roles and least privilege                               | 32        |
| 7.5 GitHub Actions OIDC instead of access keys                  | 33        |
| 7.6 Secrets, HTTPS, logs and alarms                             | 33        |
| <b>Part 8 · Scaling: 2,000 concurrent and 10,000 peak users</b> | <b>35</b> |
| 8.1 From users to requests                                      | 35        |
| 8.2 EC2 Auto Scaling group vs ECS Service Auto Scaling          | 35        |
| 8.3 Database capacity and connections                           | 36        |
| 8.4 Metrics to watch and cost trade-offs                        | 37        |
| <b>Part 9 · Deployment and operational diagrams</b>             | <b>38</b> |
| 9.1 First-time deployment                                       | 38        |

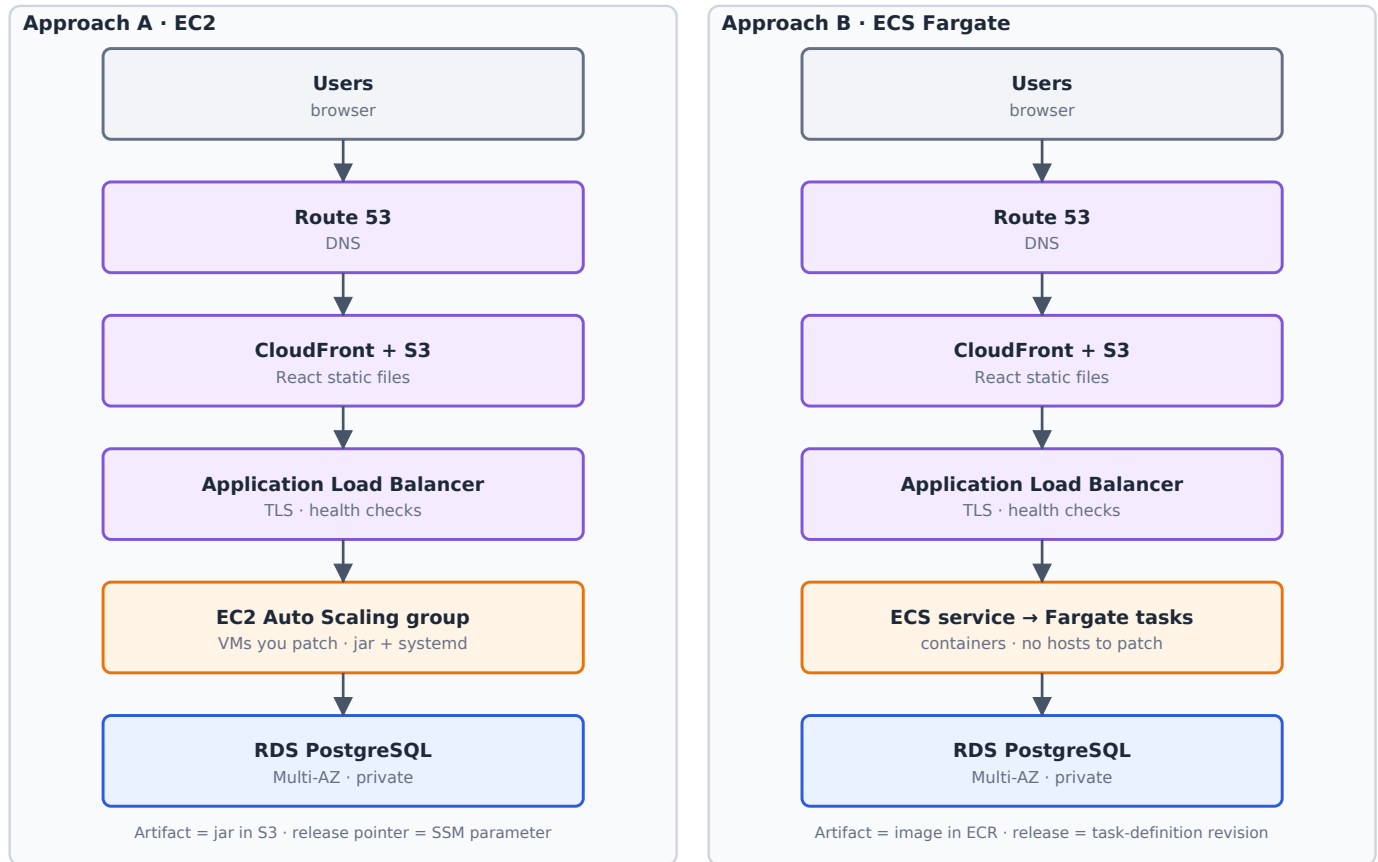
|                                                                 |           |
|-----------------------------------------------------------------|-----------|
| 9.2 Normal application update (ECS)                             | 38        |
| 9.2b Normal application update (EC2)                            | 38        |
| 9.3 Failed deployment and rollback (ECS)                        | 39        |
| 9.3b Failed deployment and rollback (EC2)                       | 39        |
| 9.4 Infrastructure change through Terraform                     | 39        |
| 9.5 Application scaling (EC2)                                   | 40        |
| 9.6 Database failure and recovery                               | 40        |
| 9.7 GitHub Actions CI/CD (ECS)                                  | 40        |
| 9.7b GitHub Actions CI/CD (EC2)                                 | 41        |
| 9.8 Request flow: browser to database                           | 41        |
| <b>Part 10 · Final comparison, recommendation and checklist</b> | <b>42</b> |
| 10.1 Recommendation                                             | 42        |
| 10.2 Files to create first (ECS path)                           | 42        |
| 10.3 State management                                           | 42        |
| 10.4 Cleanup and ongoing charges                                | 43        |
| 10.5 Final deployment checklist                                 | 43        |

**PART 1**

# Architecture overview

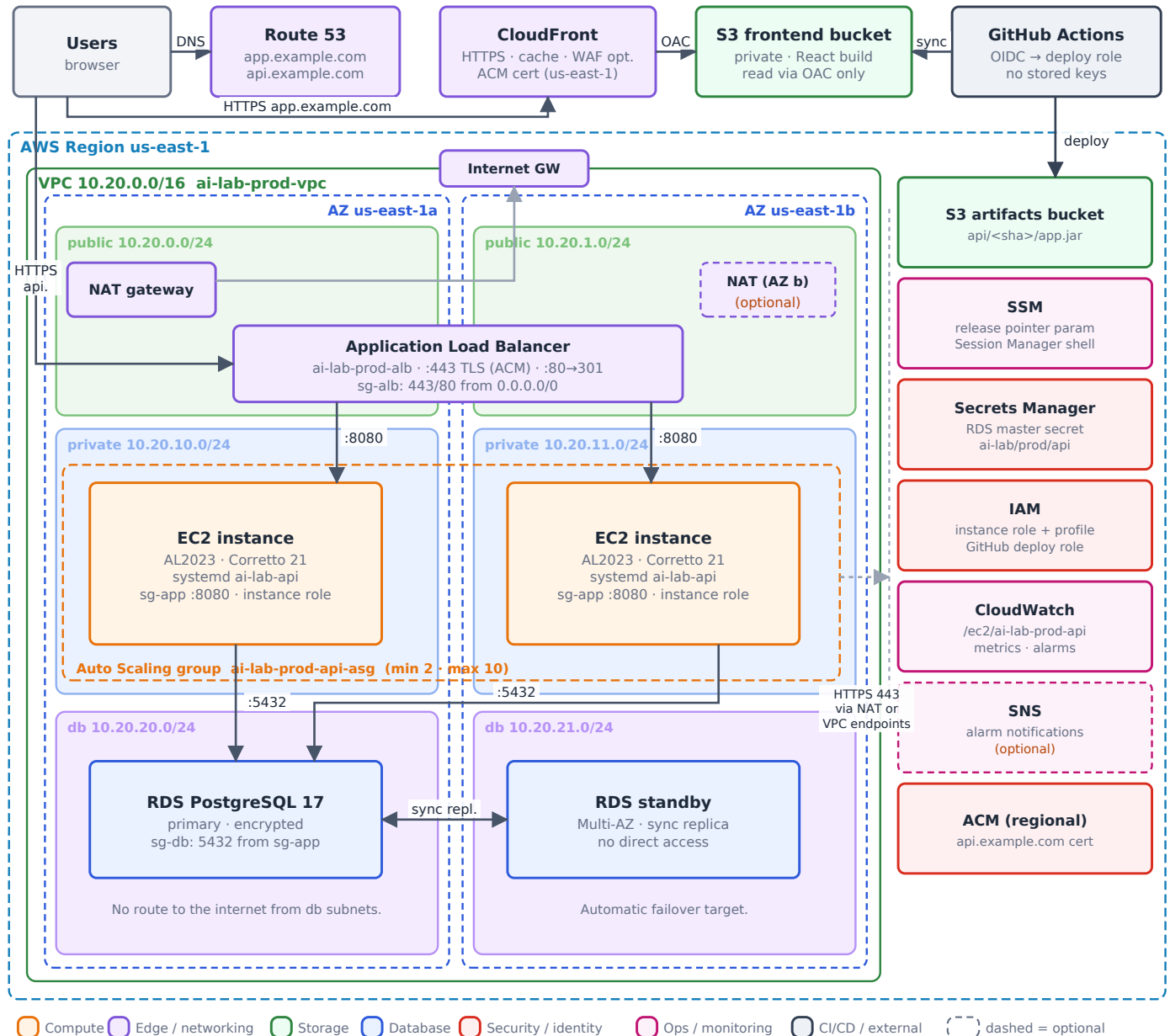
Both approaches share everything except the compute layer: the same VPC, ALB, RDS, CloudFront/S3, DNS, certificates and alarms. Only the box that runs the API changes.

## 1.1 Side by side



Same request path. A runs the API as a jar on VMs you maintain; B runs the same API as a container image on Fargate, where AWS runs the hosts.

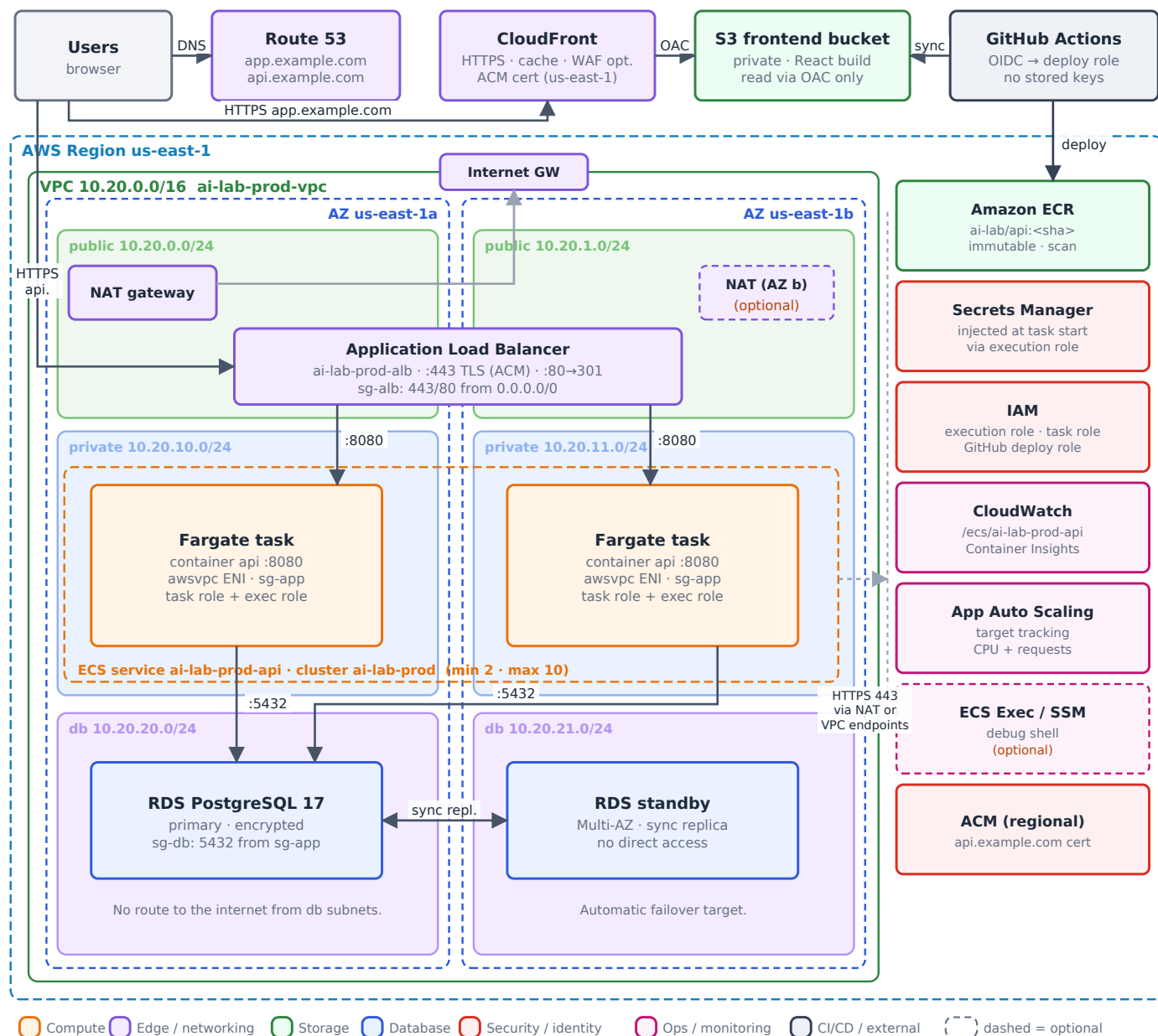
## 1.2 Approach A: EC2



Security groups chain: sg-alb → sg-app (8080) → sg-db (5432). No SSH port is open anywhere.

Users reach the React app through CloudFront and the API through the ALB. EC2 instances sit in private subnets in two AZs, receive traffic only from the ALB, and reach RDS on 5432. Deployments replace instances (Auto Scaling instance refresh). Operators get a shell through Session Manager; there is no SSH.

### 1.3 Approach B: ECS on Fargate



Security groups chain: sg-alb → sg-app (8080) → sg-db (5432). No SSH port is open anywhere.

Identical network. The ECS service keeps N copies of the task running, registers each task's private IP in the target group, injects secrets at start and rolls deployments with automatic rollback.

## 1.4 Required vs optional components

| Component                                 | A · EC2     | B · Fargate | Why / when optional                                                       |
|-------------------------------------------|-------------|-------------|---------------------------------------------------------------------------|
| VPC, 2 AZs, public + private + db subnets | Required    | Required    | ALB and Multi-AZ RDS need $\geq 2$ AZs                                    |
| Application Load Balancer + ACM cert      | Required    | Required    | TLS, health checks, spreads load                                          |
| RDS PostgreSQL (Multi-AZ)                 | Required    | Required    | Multi-AZ optional for non-prod (cost $\sim 2\times$ )                     |
| Route 53 hosted zone                      | Required*   | Required*   | *Any DNS works; Route 53 enables alias records + automated ACM validation |
| CloudFront + private S3 (React)           | Required    | Required    | Static hosting; see Part 6                                                |
| NAT gateway                               | Required*   | Required*   | *Unless all egress uses VPC endpoints and the app calls no external APIs  |
| Second NAT (one per AZ)                   | Optional    | Optional    | Removes the AZ single point of failure; doubles NAT cost                  |
| Secrets Manager                           | Required    | Required    | No passwords in code, tfvars, user-data or task definitions               |
| CloudWatch logs + alarms                  | Required    | Required    | SNS e-mail subscription optional                                          |
| Session Manager / ECS Exec                | Recommended | Optional    | Shell + DB port-forwarding without SSH                                    |
| Amazon ECR                                | —           | Required    | Image registry                                                            |
| S3 artifacts bucket + SSM release param   | Required    | —           | Jar storage + release pointer                                             |
| VPC interface endpoints, WAF, RDS Proxy   | Optional    | Optional    | Cost/security trade-offs (Parts 7–8)                                      |
| GitHub Actions + OIDC role                | Recommended | Recommended | Manual CLI deploys also work                                              |

## 1.5 EC2 vs ECS vs Fargate

| Term        | What it is                                                                                                                                       | Analogy                     |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------|
| Amazon EC2  | Virtual machines. You choose the AMI, patch the OS, install the runtime and run processes.                                                       | Renting servers             |
| Amazon ECS  | Container orchestrator: keeps N copies of a container running, rolls deployments, integrates with ALB, IAM, logs. Free; you pay for the compute. | The scheduler               |
| AWS Fargate | Serverless compute for ECS tasks: each task gets its own AWS-managed micro-VM. No instances to see or patch.                                     | The engine room, run by AWS |
| ECS on EC2  | ECS schedules containers onto EC2 instances that you own (capacity provider + ECS-optimised AMI). More control, more work.                       | Scheduler + your servers    |

|                                      | EC2 (VMs) | ECS on EC2 | ECS on Fargate |
|--------------------------------------|-----------|------------|----------------|
| Application code + dependencies      | you       | you        | you            |
| Container image / runtime config     | n/a (jar) | you        | you            |
| Scheduling, placement, scaling rules | you: ASG  | shared     | shared         |
| Container runtime + ECS agent        | n/a       | you*       | AWS            |
| Guest OS, patches, AMI, SSM agent    | you       | you        | AWS            |
| Host capacity (instance fleet)       | you: ASG  | you        | AWS            |
| Hypervisor, hardware, data centre    | AWS       | AWS        | AWS            |

you\* = you choose/update the ECS-optimised AMI that contains the agent. shared = you set the rules, AWS executes them.

Who maintains which layer. Fargate moves the OS, agent and host-capacity layers to AWS. In every model you own the application, its IAM permissions, network rules, data and secrets.

**Your responsibility** In both approaches you own: IAM least privilege, security-group rules, secrets lifecycle, RDS sizing, backups/restore testing, alarms and on-call, cost, and the application's own vulnerabilities. AWS owns the physical/virtual infrastructure, the managed control planes (ALB, RDS engine operations, ECS scheduler, CloudFront).

## PART 2

# Inventory of every configuration file

Each approach is one Terraform directory plus the files that CI and the instances/containers read. Files with the same name are identical in both directories, so you only ever learn them once.

## 2.1 Directory trees

### A · EC2

```
repo/
├── .github/workflows/deploy-ec2.yml      # CI/CD (GitHub Actions)
├── services/ai-engineering-api/         # backend (built to app.jar)
├── frontend/                           # React app (built to dist/)
├── infrastructure/aws/ec2/              # Terraform root module
│   ├── versions.tf                    providers.tf          variables.tf
│   ├── terraform.tfvars.example       # copy → terraform.tfvars (git-ignored)
│   ├── network.tf                    security-groups.tf    iam.tf
│   ├── ec2.tf                        load-balancer.tf      rds.tf
│   ├── dns-acm.tf                    frontend.tf          github-oidc.tf
│   ├── monitoring.tf                 outputs.tf
│   └── templates/
│       ├── user-data.sh.tftpl        # bootstrap, rendered by templatefile()
│       └── ai-lab-api.service         # systemd unit, embedded in user-data
└── scripts/deploy-release.sh          # deploy + rollback (CI and humans)
```

### B · ECS Fargate

```
repo/
├── .github/workflows/deploy-ecs.yml      # CI/CD (GitHub Actions)
├── services/ai-engineering-api/
│   ├── Dockerfile                     # container packaging input
│   └── .dockerignore
├── frontend/
├── infrastructure/aws/ecs/              # Terraform root module
│   ├── versions.tf                    providers.tf          variables.tf
│   ├── terraform.tfvars.example
│   ├── network.tf                    security-groups.tf    iam.tf
│   ├── ecr.tf                        ecs-cluster.tf        ecs-task-definition.tf
│   ├── ecs-service.tf                 autoscaling.tf        load-balancer.tf
│   ├── rds.tf                        dns-acm.tf            frontend.tf
│   ├── github-oidc.tf                 monitoring.tf         outputs.tf
└── (reference only, not committed: ecs-task-definition.json = what AWS stores)
```

**Note** Identical in both: `providers.tf`, `network.tf`, `security-groups.tf`, `load-balancer.tf`, `rds.tf`, `dns-acm.tf`, `frontend.tf`, `github-oidc.tf`, `monitoring.tf`. `load-balancer.tf` reads `local.lb_target_type`, which is "instance" in `ec2.tf` and "ip" in `ecs-service.tf`. Terraform reads every `*.tf` file in a directory as one configuration; splitting into files is purely for readability.

## 2.2 Approach A file inventory

| File                                      | What it defines · why it exists                                                                                        | Read by                     | Feeds / depends                 | Status        |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------|-----------------------------|---------------------------------|---------------|
| <code>versions.tf</code>                  | Terraform/provider version pins + S3 remote state ( <code>use_lockfile</code> ). Reproducible, shared state.           | terraform init              | every resource; state in S3     | Required      |
| <code>providers.tf</code>                 | AWS provider, default tags, <code>us_east_1</code> alias (CloudFront cert), shared <code>locals</code> (names, FQDNs). | Terraform                   | all files                       | Required      |
| <code>variables.tf</code>                 | Every input with type, default, validation.                                                                            | Terraform                   | all files                       | Required      |
| <code>terraform.tfvars.example</code>     | Example values. Copy to <code>terraform.tfvars</code> (git-ignored). No secrets.                                       | You → Terraform             | variables.tf                    | Env-dependent |
| <code>network.tf</code>                   | VPC, 3 subnet tiers × 2 AZs, IGW, NAT, route tables, free S3 gateway endpoint.                                         | Terraform                   | SGs, ALB, ASG, RDS              | Required      |
| <code>security-groups.tf</code>           | Only allowed paths: internet→ALB→app→DB; app→443.                                                                      | Terraform                   | ALB, launch template, RDS       | Required      |
| <code>iam.tf</code>                       | Instance role + profile (SSM, CW agent, read jar/param/secrets) and CI deploy permissions.                             | Terraform; IAM evaluates    | launch template, github-oidc.tf | Required      |
| <code>ec2.tf</code>                       | Artifacts bucket, release pointer param, log group, AMI lookup, launch template, ASG, scaling.                         | Terraform                   | templates/, iam, ALB TG         | Required      |
| <code>templates/user-data.sh.tftpl</code> | First-boot bootstrap: packages, user, env files, secret fetcher, deploy script, CW agent.                              | templatefile() → cloud-init | launch template                 | Required      |
| <code>templates/ai-lab-api.service</code> | systemd unit: run/restart java as <code>ailab</code> , env files, hardening, log file.                                 | systemd                     | user-data                       | Required      |
| <code>load-balancer.tf</code>             | ALB, target group + health check, HTTP→HTTPS redirect, HTTPS listener.                                                 | Terraform                   | dns-acm, ASG                    | Required      |
| <code>rds.tf</code>                       | Subnet group, parameter group (force SSL), Multi-AZ instance, managed secret; app-secret lookup.                       | Terraform                   | user-data, iam                  | Required      |
| <code>dns-acm.tf</code>                   | ACM certs (regional + us-east-1), DNS validation, alias records.                                                       | Terraform                   | ALB, CloudFront                 | Required*     |
| <code>frontend.tf</code>                  | Private S3 bucket, OAC, CloudFront, SPA fallback, bucket policy.                                                       | Terraform                   | dns-acm, CI                     | Required      |
| <code>github-oidc.tf</code>               | OIDC provider, deploy role trust (repo + environment), frontend publish policy.                                        | Terraform; STS              | workflow                        | Recommended   |
| <code>monitoring.tf</code>                | SNS topic (+ optional e-mail), ALB/RDS alarms.                                                                         | Terraform; CloudWatch       | on-call                         | Recommended   |
| <code>outputs.tf</code>                   | Values CI and operators need (role ARN, buckets, ASG name...).                                                         | terraform output            | GitHub variables                | Required      |
| <code>scripts/deploy-release.sh</code>    | Upload jar, move release pointer, instance refresh, wait, restore pointer on failure.                                  | CI / operator               | S3, SSM, ASG                    | Required      |
| <code>deploy-ec2.yml</code>               | Test, package, deploy API, then publish React; manual rollback input.                                                  | GitHub Actions              | deploy role, script             | Recommended   |

**Note Nginx is not used:** the ALB already terminates TLS, redirects HTTP and routes to port 8080, so a local reverse proxy would add a hop and config without benefit. **Ansible is not included:** instances are immutable (replaced on every release), so user-data + Terraform cover configuration. Ansible earns its place for long-lived, mutable fleets. The **CloudWatch agent config** lives inside user-data, not as a separate file.

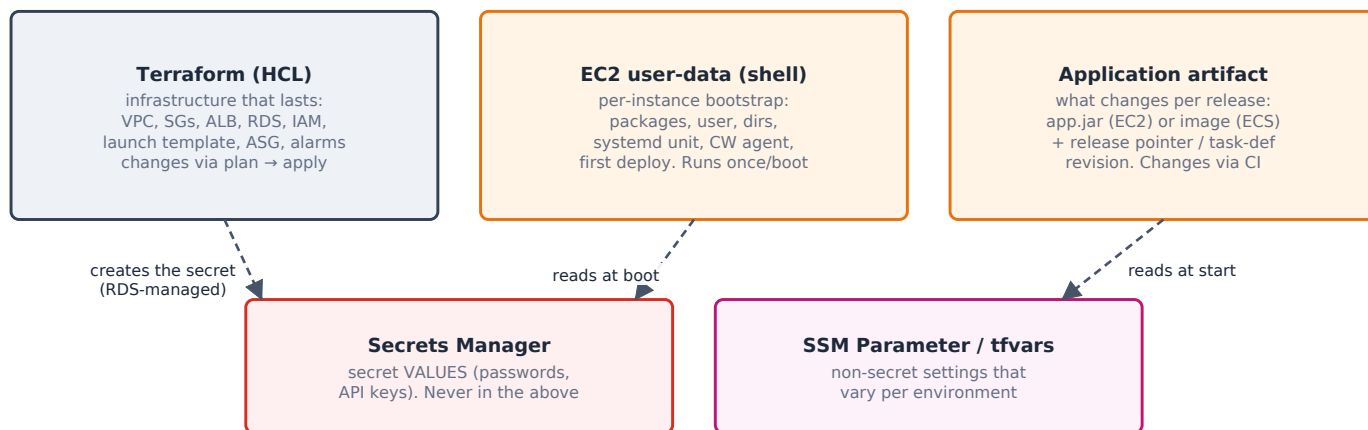
## 2.3 Approach B file inventory

| File                                                                                                                                                                            | What it defines · why it exists                                                                                                   | Read by          | Feeds / depends       | Status      |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|------------------|-----------------------|-------------|
| <code>versions.tf</code> ...<br><code>variables.tf</code> , <code>tfvars</code>                                                                                                 | As in A (state key <code>prod/ecs/...</code> ; ECS variables: task size, counts, <code>initial_image_tag</code> ).                | Terraform        | all                   | Required    |
| <code>network.tf</code> ,<br><code>security-groups.tf</code>                                                                                                                    | Identical to A. <code>sg-app</code> is attached to each task ENI.                                                                 | Terraform        | service, ALB, RDS     | Required    |
| <code>iam.tf</code>                                                                                                                                                             | Task <b>execution</b> role (pull, secrets, logs), task role (app), CI deploy permissions incl. scoped <code>iam:PassRole</code> . | Terraform; IAM   | task definition, CI   | Required    |
| <code>ecr.tf</code>                                                                                                                                                             | Private repository, immutable tags, scan on push, lifecycle (keep 30).                                                            | Terraform        | CI push, task def     | Required    |
| <code>ecs-cluster.tf</code>                                                                                                                                                     | Cluster (Container Insights) + log group <code>/ecs/ai-lab-prod-api</code> .                                                      | Terraform        | service, task def     | Required    |
| <code>ecs-task-definition.tf</code>                                                                                                                                             | Blueprint: image, CPU/memory, port, env, <b>secrets</b> (ARNs), health check, <code>awslogs</code> .                              | Terraform → ECS  | service; CI copies it | Required    |
| <code>ecs-service.tf</code>                                                                                                                                                     | Desired count, subnets/SG, target group, rolling params, circuit breaker, alarm rollback.                                         | Terraform → ECS  | autoscaling, ALB      | Required    |
| <code>autoscaling.tf</code>                                                                                                                                                     | Scalable target (min/max) + CPU and request-count target tracking.                                                                | App Auto Scaling | service               | Required    |
| <code>load-balancer.tf</code> ,<br><code>rds.tf</code> , <code>dns-acm.tf</code> ,<br><code>frontend.tf</code> ,<br><code>github-oidc.tf</code> ,<br><code>monitoring.tf</code> | Identical to A.                                                                                                                   | Terraform        | —                     | As in A     |
| <code>outputs.tf</code>                                                                                                                                                         | ECR URL, cluster/service names, role ARN, bucket, distribution ID.                                                                | terraform output | GitHub variables      | Required    |
| <code>Dockerfile</code>                                                                                                                                                         | Two-stage build → non-root JRE image with curl for the health check.                                                              | Docker/BuildKit  | ECR image             | Required    |
| <code>.dockerignore</code>                                                                                                                                                      | Keeps build output, Git data and <code>.env</code> files out of the build context.                                                | Docker           | Dockerfile            | Required    |
| <code>deploy-ecs.yml</code>                                                                                                                                                     | Test → build/push → render task def → update service (waits) → publish React.                                                     | GitHub Actions   | ECR, ECS, S3          | Recommended |

### Alternative ways to express the same thing: pick ONE

| Concern                | Chosen here                                                                                                                                                                             | Alternative (don't keep both)                                                                                                                        |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Task definition        | <code>ecs-task-definition.tf</code> is the source of truth. CI downloads the latest revision ( <code>describe-task-definition</code> ), swaps only the image, registers a new revision. | Commit <code>taskdef.json</code> and let CI register it; Terraform then only bootstraps the service. Simpler CI, but config lives outside Terraform. |
| Service ↔ CI ownership | <code>ignore_changes = [task_definition, desired_count]</code> so Terraform never reverts CI deploys or autoscaling.                                                                    | CI runs <code>terraform apply -var image_tag=&lt;sha&gt;</code> for every deploy (needs state access + broad CI permissions).                        |
| Health check           | ALB target-group check (decides traffic) + container <code>healthCheck</code> (decides task health).                                                                                    | ALB check only: also valid; ECS still replaces tasks the ALB marks unhealthy.                                                                        |
| Secrets                | RDS-managed master secret + CLI-created app secret, referenced by ARN.                                                                                                                  | SSM Parameter Store SecureString ( <code>valueFrom</code> = parameter ARN).                                                                          |
| Docker Compose         | Local development only (the repo's <code>docker-compose.yml</code> runs PostgreSQL locally).                                                                                            | Not an AWS production orchestrator; ECS replaces it.                                                                                                 |

## 2.4 Where does a setting belong?



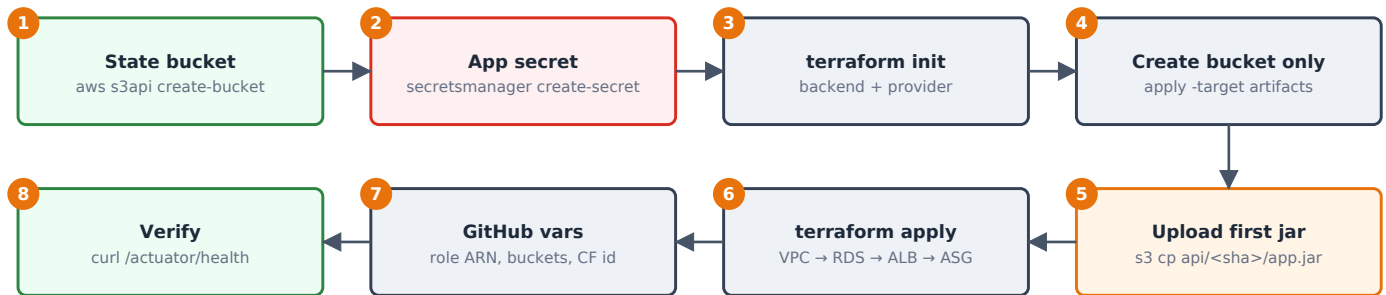
Rule of thumb: if it changes every release → artifact/CI. Per instance boot → user-data. Rarely and shared → Terraform.

Terraform for long-lived infrastructure, user-data for per-instance bootstrap, the artifact for per-release changes, Secrets Manager for secret values, SSM/tfvars for per-environment non-secret settings.

## PART 3

# EC2 deployment walkthrough

Run from a workstation with admin-level bootstrap credentials (an SSO session is ideal). All commands use AWS CLI v2 syntax, run from `infrastructure/aws/ec2/` unless stated otherwise.



First-time deployment order. Steps 1-2 create things Terraform reads but should not own (state bucket, secret values). Step 4 is the only targeted apply: instances need a jar to exist on first boot.

## 0 Prerequisites

|                     |                                                                   |
|---------------------|-------------------------------------------------------------------|
| <b>AWS resource</b> | none                                                              |
| <b>Expected</b>     | Your account ID and ARN are printed; tools report their versions. |

shell

```
aws --version           # aws-cli/2.x
terraform version      # >= 1.10 (S3 native locking)
session-manager-plugin --version # for aws ssm start-session
aws sts get-caller-identity # confirm the right account
aws route53 list-hosted-zones-by-name --dns-name example.com --max-items 1
```

| If it fails: symptom         | Likely cause → how to diagnose / fix                                                                        |
|------------------------------|-------------------------------------------------------------------------------------------------------------|
| Unable to locate credentials | No profile/SSO session → <code>aws sso login --profile &lt;p&gt;</code> ; export <code>AWS_PROFILE</code> . |
| Hosted zone missing          | Create/delegate the public zone first; ACM DNS validation needs it.                                         |

## 1 Create the Terraform state bucket (once per account)

|                     |                                                                                                                 |
|---------------------|-----------------------------------------------------------------------------------------------------------------|
| <b>AWS resource</b> | S3 bucket <code>ai-lab-tfstate-11112223333</code> (versioned, public access blocked; default SSE-S3 encryption) |
| <b>Expected</b>     | <code>create-bucket</code> returns a <code>Location</code> ; the other commands print nothing.                  |

shell

```
STATE_BUCKET=ai-lab-tfstate-11112223333 # REPLACE account ID
aws s3api create-bucket --bucket "$STATE_BUCKET" --region us-east-1
aws s3api put-bucket-versioning --bucket "$STATE_BUCKET" \
  --versioning-configuration Status=Enabled
aws s3api put-public-access-block --bucket "$STATE_BUCKET" \
  --public-access-block-configuration \
    BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
```

| If it fails: symptom               | Likely cause → how to diagnose / fix                                                                 |
|------------------------------------|------------------------------------------------------------------------------------------------------|
| BucketAlreadyExists                | Bucket names are global → change the suffix and update <code>versions.tf</code> .                    |
| IllegalLocationConstraintException | Outside us-east-1 add <code>--create-bucket-configuration LocationConstraint=&lt;region&gt;</code> . |

**Security** Terraform state can contain sensitive values (resource attributes, generated values, anything a `*_secret_version` resource holds). Keep it only in this versioned, private bucket. Never commit `*.tfstate*`, saved plan files (`tfplan`) or `terraform.tfvars`. Do commit `.terraform.lock.hcl`.

## 2 Create the application secret (values never touch Terraform)

|                     |                                                                           |
|---------------------|---------------------------------------------------------------------------|
| <b>File</b>         | <code>app-secret.json</code> (temporary, local, deleted afterwards)       |
| <b>AWS resource</b> | Secrets Manager secret <code>ai-lab/prod/api</code>                       |
| <b>Expected</b>     | JSON with <code>ARN</code> , <code>Name</code> , <code>VersionId</code> . |

shell

```
umask 077
cat > app-secret.json <<'EOF'
{ "OPENAI_API_KEY": "REPLACE-or-empty-string" }
EOF
aws secretsmanager create-secret --name ai-lab/prod/api \
  --description "ai-lab prod API runtime secrets" \
  --secret-string file://app-secret.json
rm -f app-secret.json
```

| If it fails: symptom      | Likely cause → how to diagnose / fix                                                                                                   |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| ResourceExistsException   | Already created → update with <code>aws secretsmanager put-secret-value --secret-id ai-lab/prod/api --secret-string file:///...</code> |
| ...scheduled for deletion | <code>aws secretsmanager restore-secret --secret-id ai-lab/prod/api.</code>                                                            |

### 3 Write configuration and initialise Terraform

|              |                                                                                                                 |
|--------------|-----------------------------------------------------------------------------------------------------------------|
| File         | <code>versions.tf</code> (backend), <code>terraform.tfvars</code> (from <code>terraform.tfvars.example</code> ) |
| AWS resource | none yet (downloads provider, connects to state bucket)                                                         |
| Expected     | Terraform has been successfully initialized!                                                                    |

`versions.tf` backend + pins

```
terraform {
  required_version = ">= 1.10.0" # S3 native state locking (use_lockfile)

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }

  backend "s3" {
    bucket  = "ai-lab-tfstate-11112223333" # REPLACE: your state bucket (account ID suffix)
    key     = "prod/ec2/terraform.tfstate"
    region  = "us-east-1"
    encrypt = true
    use_lockfile = true # lock file in S3; no DynamoDB table needed
  }
}
```

shell

```
cp terraform.tfvars.example terraform.tfvars # then edit domain_name, github_repo, ...
terraform init
terraform fmt -check && terraform validate
```

| If it fails: symptom             | Likely cause → how to diagnose / fix                                                                         |
|----------------------------------|--------------------------------------------------------------------------------------------------------------|
| AccessDenied on the state bucket | Wrong account/profile, or bucket name differs from <code>versions.tf</code> .                                |
| Backend configuration changed    | <code>terraform init -reconfigure</code> (new backend) or <code>-migrate-state</code> (move existing state). |

### 4 Bootstrap: artifact bucket + first release

|              |                                                                                                                    |
|--------------|--------------------------------------------------------------------------------------------------------------------|
| AWS resource | S3 bucket <code>ai-lab-prod-artifacts-11112223333</code> (versioned) + object <code>api/&lt;sha&gt;/app.jar</code> |
| Expected     | Apply complete! Resources: 4 added then an <code>upload:</code> line.                                              |

shell

```
terraform apply -target=aws_s3_bucket.artifacts \
  -target=aws_s3_bucket_public_access_block.artifacts \
  -target=aws_s3_bucket_versioning.artifacts \
  -target=aws_s3_bucket_lifecycle_configuration.artifacts
# build the jar with the project's own build (CI does the same)
(cd ../../../../services/ai-engineering-api && ./mvnw -B package -DskipTests)
SHA=$(git rev-parse HEAD)
aws s3 cp ../../../../services/ai-engineering-api/target/*.jar \
  "s3://ai-lab-prod-artifacts-11112223333/api/$SHA/app.jar"
# set initial_release = "<that SHA>" in terraform.tfvars
```

| If it fails: symptom  | Likely cause → how to diagnose / fix                                                                                              |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| -target warning       | Expected: targeted applies are for bootstrapping only; the next full apply converges everything.                                  |
| Multiple jars matched | Copy the exact file name from <code>target/</code> (the <code>*.jar.original</code> file is not a match for <code>*.jar</code> ). |

### 5 Review the plan

|              |                                                                                                             |
|--------------|-------------------------------------------------------------------------------------------------------------|
| AWS resource | none (read-only)                                                                                            |
| Expected     | Plan: N to add, 0 to change, 0 to destroy. Read every resource; nothing should be destroyed on a first run. |

shell

```
terraform plan -out=tfplan
terraform show -no-color tfplan | grep -E '^ #' | sort | head -80 # list of resources
```

| If it fails: symptom                   | Likely cause → how to diagnose / fix                       |
|----------------------------------------|------------------------------------------------------------|
| no matching Route 53 Hosted Zone found | <code>domain_name</code> must equal the zone name exactly. |
| couldn't find resource (secret)        | Step 2 not done, or a different region.                    |
| Errors about quotas                    | Check Service Quotas (e.g. Elastic IPs, VPCs per region).  |

## 6 Create the infrastructure

|                     |                                                                                                                               |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <b>AWS resource</b> | VPC, subnets, NAT, SGs, IAM, RDS (+ managed secret), ACM, ALB, launch template, ASG, CloudFront, S3, Route 53 records, alarms |
| <b>Expected</b>     | <b>Apply complete!</b> and outputs. Expect roughly 15-40 min: RDS Multi-AZ and CloudFront dominate.                           |

shell

```
terraform apply tfplan
terraform output
```

| If it fails: symptom                                         | Likely cause → how to diagnose / fix                                                                                                                                        |
|--------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Stuck on <code>aws_acm_certificate_validation</code>         | Validation CNAME not resolvable → zone not delegated at the registrar ( <b>dig NS example.com</b> ).                                                                        |
| <code>InvalidParameterCombination</code> (RDS version/class) | <code>aws rds describe-orderable-db-instance-options --engine postgres --engine-version 17 --db-instance-class db.t4g.medium --query 'OrderableDBInstanceOptions[0]'</code> |
| ASG <b>Waiting for capacity</b>                              | Instances failing ELB checks → continue with steps 7-8 to diagnose.                                                                                                         |

## 7 Connect securely and confirm the bootstrap

|                     |                                                                                                                 |
|---------------------|-----------------------------------------------------------------------------------------------------------------|
| <b>File</b>         | <code>templates/user-data.sh.tftpl</code> , <code>templates/ai-lab-api.service</code>                           |
| <b>AWS resource</b> | SSM managed instances; no inbound port is opened                                                                |
| <b>Expected</b>     | Instances listed as <b>Online</b> ; <b>user-data done</b> at the end of the log; unit <b>active (running)</b> . |

`templates/ai-lab-api.service` excerpt

```
# /etc/systemd/system/ai-lab-api.service - keeps the API process running on the instance.
# Installed by user-data. Secrets are fetched at every (re)start into a tmpfs file.
[Unit]
Description=ai-lab API
Wants=network-online.target
After=network-online.target

[Service]
Type=simple
User=ailab
Group=ailab
WorkingDirectory=/opt/ai-lab/current

# Non-secret config (written once by user-data) + secrets (written by ExecStartPre).
EnvironmentFile=/etc/ai-lab/api.env
EnvironmentFile=-/run/ai-lab/secrets.env
# "+" runs this one command as root so it can write /run/ai-lab with mode 0600.
ExecStartPre+=/usr/local/bin/ai-lab-fetch-secrets

ExecStart=/usr/bin/java -XX:MaxRAMPercentage=75 -XX:+ExitOnOutOfMemoryError -jar /opt/ai-lab/current/app.jar
# JVM exits with 143 after SIGTERM: treat as a clean stop.
SuccessExitStatus=143
Restart=on-failure
RestartSec=5
TimeoutStopSec=30

StandardOutput=append:/var/log/ai-lab/api.log
StandardError=append:/var/log/ai-lab/api.log

# Hardening
NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=strict
ProtectHome=true
ReadWritePaths=/opt/ai-lab
LimitNOFILE=65536
```

shell

```
aws ssm describe-instance-information \
  --query 'InstanceInformationList[].[InstanceId,PingStatus]' --output table
aws ssm start-session --target i-0123456789abcdef0 # REPLACE instance ID
# inside the session:
sudo tail -n 30 /var/log/user-data.log
systemctl status ai-lab-api --no-pager
curl -s localhost:8080/actuator/health
```

| If it fails: symptom                                   | Likely cause → how to diagnose / fix                                                                                                                |
|--------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Instance not in SSM list                               | No outbound 443 (NAT/route/SG) or role missing <code>AmazonSSMManagedInstanceCore</code> . Check: EC2 console → Actions → Monitor → Get system log. |
| <code>dnf</code> errors in user-data log               | No internet egress. Verify the private route table <code>0.0.0.0/0 → nat-....</code>                                                                |
| <code>AccessDeniedException</code> from secretsmanager | Instance role lacks the secret ARN, or the secret uses a customer KMS key (add <code>kms:Decrypt</code> ).                                          |
| Unit restarts in a loop                                | <code>sudo tail -n 100 /var/log/ai-lab/api.log</code> (DB unreachable? wrong env?).                                                                 |

## 8 Check ALB health checks

|                     |                                                                         |
|---------------------|-------------------------------------------------------------------------|
| <b>File</b>         | <code>load-balancer.tf</code> (target group <code>health_check</code> ) |
| <b>AWS resource</b> | Target group <code>ai-lab-prod-api-tg</code>                            |
| <b>Expected</b>     | Every target <code>healthy</code> .                                     |

shell

```
TG=$(aws elbv2 describe-target-groups --names ai-lab-prod-api-tg \
  --query 'TargetGroups[0].TargetGroupArn' --output text)
aws elbv2 describe-target-health --target-group-arn "$TG" \
  --query 'TargetHealthDescriptions[].[Target.Id,TargetHealth.State,TargetHealth.Reason]' --output table
```

| If it fails: symptom                     | Likely cause → how to diagnose / fix                                           |
|------------------------------------------|--------------------------------------------------------------------------------|
| <code>Target.Timeout</code>              | SG path broken (sg-alb egress / sg-app ingress 8080) or app not listening yet. |
| <code>Target.ResponseCodeMismatch</code> | Health path returns non-200 (e.g. 503 when the DB is down) → check app log.    |
| <code>Elb.InitialHealthChecking</code>   | Normal for ~30-60 s after registration.                                        |

## 9 Verify HTTPS and DNS end to end

|                     |                                                                                                                                         |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <b>AWS resource</b> | ACM certificates, Route 53 alias records, CloudFront                                                                                    |
| <b>Expected</b>     | Cert <code>ISSUED</code> ; DNS resolves; health returns <code>{"status":"UP"...}</code> ; / returns the React <code>index.html</code> . |

shell

```
aws acm list-certificates --region us-east-1 \
  --query 'CertificateSummaryList[].[DomainName,Status]' --output table
dig +short api.example.com
curl -sS https://api.example.com/actuator/health
curl -sSI http://api.example.com | head -3 # expect 301 to https
curl -sSI https://app.example.com/some/deep/link | head -1 # expect 200 (SPA fallback)
```

| If it fails: symptom             | Likely cause → how to diagnose / fix                                           |
|----------------------------------|--------------------------------------------------------------------------------|
| TLS name mismatch                | Calling the ALB DNS name directly; use the custom domain.                      |
| CloudFront <code>403</code> on / | Bucket empty (frontend not published yet) or bucket policy SourceArn mismatch. |

## 10 Admin access to the database (port forwarding, no public DB)

|                     |                                                                                    |
|---------------------|------------------------------------------------------------------------------------|
| <b>AWS resource</b> | SSM session through an API instance to RDS:5432                                    |
| <b>Expected</b>     | <code>Waiting for connections...</code> ; then <code>psql</code> connects locally. |

shell

```
DB_HOST=$(terraform output -raw db_endpoint)
aws ssm start-session --target i-0I23456789abcdef0 \
  --document-name AWS-StartPortForwardingSessionToRemoteHost \
  --parameters ["host\":[\"$DB_HOST\"],\"portNumber\":[\"5432\"],\"localPortNumber\":[\"15432\"]}"
# second terminal: fetch the password without echoing it into history
export PGPASSWORD=$(aws secretsmanager get-secret-value \
  --secret-id "$(terraform output -raw db_secret_arn)" --query SecretString --output text | jq -r .password)
psql "host=localhost port=15432 dbname=ai_lab user=ai_lab_admin sslmode=require" -c 'select version();'
```

| If it fails: symptom                                | Likely cause → how to diagnose / fix                              |
|-----------------------------------------------------|-------------------------------------------------------------------|
| <code>SessionManagerPlugin</code> is not found      | Install the Session Manager plugin locally.                       |
| no <code>pg_hba.conf</code> entry ... no encryption | <code>rds.force_ssl=1</code> → use <code>sslmode=require</code> . |

## 11 Wire up GitHub Actions

|                     |                                                                                                              |
|---------------------|--------------------------------------------------------------------------------------------------------------|
| <b>File</b>         | <code>.github/workflows/deploy-ec2.yml</code> , <code>github-oidc.tf</code>                                  |
| <b>AWS resource</b> | GitHub environment <code>production</code> + repository/environment variables (not secrets: none are needed) |
| <b>Expected</b>     | Variables listed in Settings → Environments → <code>production</code> .                                      |

shell

```
gh api -X PUT repos/OWNER/ai-engineering-lab/environments/production # creates the environment
for v in AWS_DEPLOY_ROLE_ARN:github_deploy_role_arn ARTIFACT_BUCKET:artifact_bucket \
  FRONTEND_BUCKET:frontend_bucket CLOUDFRONT_DISTRIBUTION_ID:cloudfront_distribution_id; do
  gh variable set "${v%:*}" --env production --body "$(terraform output -raw ${v##*:})"
done
```

| If it fails: symptom                                                 | Likely cause → how to diagnose / fix                                                                                                     |
|----------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Not authorized to perform <code>sts:AssumeRoleWithWebIdentity</code> | Job lacks environment: <code>production</code> , wrong <code>github_repo</code> , or missing permissions: <code>id-token: write</code> . |

## 12 Monitoring

|                     |                                                                                                   |
|---------------------|---------------------------------------------------------------------------------------------------|
| <b>File</b>         | <code>monitoring.tf</code> , CloudWatch agent JSON inside user-data                               |
| <b>AWS resource</b> | SNS topic <code>ai-lab-prod-alerts</code> , 6 alarms, log group <code>/ec2/ai-lab-prod-api</code> |
| <b>Expected</b>     | Alarms in <b>OK</b> (or <b>INSUFFICIENT_DATA</b> before traffic); logs streaming.                 |

### shell

```
aws cloudwatch describe-alarms --alarm-name-prefix ai-lab-prod \
  --query 'MetricAlarms[][AlarmName,StateValue]' --output table
aws logs tail /ec2/ai-lab-prod-api --follow --since 10m
```

| If it fails: symptom | Likely cause → how to diagnose / fix                                                           |
|----------------------|------------------------------------------------------------------------------------------------|
| No e-mails           | Confirm the SNS subscription link sent to <code>alert_email</code> .                           |
| Empty log group      | CW agent not running: <code>sudo systemctl status amazon-cloudwatch-agent</code> in a session. |

## 13 Test recovery and scaling

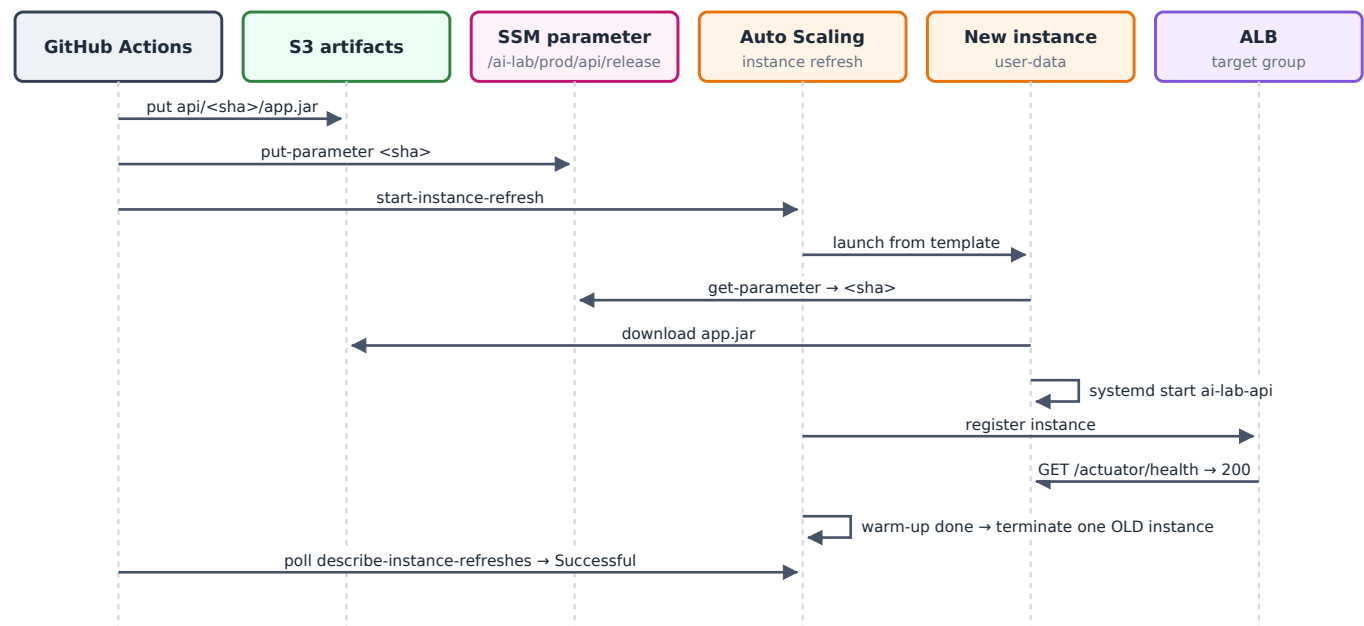
|                     |                                                                                                                  |
|---------------------|------------------------------------------------------------------------------------------------------------------|
| <b>AWS resource</b> | Auto Scaling group <code>ai-lab-prod-api-asg</code>                                                              |
| <b>Expected</b>     | A replacement instance launches and becomes healthy; desired capacity changes are reflected in the target group. |

### shell

```
aws autoscaling terminate-instance-in-auto-scaling-group \
  --instance-id i-0123456789abcdef0 --no-should-decrement-desired-capacity
aws autoscaling describe-scaling-activities --auto-scaling-group-name ai-lab-prod-api-asg --max-items 3
aws autoscaling set-desired-capacity --auto-scaling-group-name ai-lab-prod-api-asg --desired-capacity 3
```

| If it fails: symptom      | Likely cause → how to diagnose / fix                                                      |
|---------------------------|-------------------------------------------------------------------------------------------|
| Replacement keeps failing | Same root causes as steps 7-8; <code>describe-scaling-activities</code> shows the reason. |

## 3.1 Deploying an application update



A release is three moves: upload `api/<sha>/app.jar`, point `/ai-lab/prod/api/release` at it, start an instance refresh. Instances are replaced, never patched in place, so every instance is built the same way.

### scripts/deploy-release.sh called by `deploy-ec2.yml`

```
#!/usr/bin/env bash
# deploy-release.sh - release (or roll back) the API on the EC2 Auto Scaling group.
# Used by GitHub Actions and by operators. Requires AWS credentials in the environment.
#
# Deploy a new build : scripts/deploy-release.sh <release-id> path/to/app.jar
# Roll back          : scripts/deploy-release.sh <previous-release-id>
#                    (jar already in S3, so no upload)
set -euo pipefail

release="${1:?usage: deploy-release.sh <release-id> [app.jar]}"
jar="${2:-}"

: "${AWS_REGION:?set AWS_REGION}"
: "${ARTIFACT_BUCKET:?set ARTIFACT_BUCKET}" # ai-lab-prod-artifacts-<account-id>
: "${ASG_NAME:?set ASG_NAME}"              # ai-lab-prod-api-asg
: "${RELEASE_PARAM:?set RELEASE_PARAM}"    # /ai-lab/prod/api/release

key="api/${release}/app.jar"
```

**scripts/deploy-release.sh** called by **deploy-ec2.yml**

```

if [[ -n "$jar" ]]; then
  echo "Uploading $jar to s3://$ARTIFACT_BUCKET/$key"
  aws s3 cp "$jar" "s3://$ARTIFACT_BUCKET/$key" --region "$AWS_REGION"
else
  echo "Checking that s3://$ARTIFACT_BUCKET/$key exists"
  aws s3api head-object --bucket "$ARTIFACT_BUCKET" --key "$key" --region "$AWS_REGION" >/dev/null
fi

previous=$(aws ssm get-parameter --name "$RELEASE_PARAM" --region "$AWS_REGION" \
  --query Parameter.Value --output text)
echo "Release pointer: $previous -> $release"
aws ssm put-parameter --name "$RELEASE_PARAM" --value "$release" --type String \
  --overwrite --region "$AWS_REGION" >/dev/null

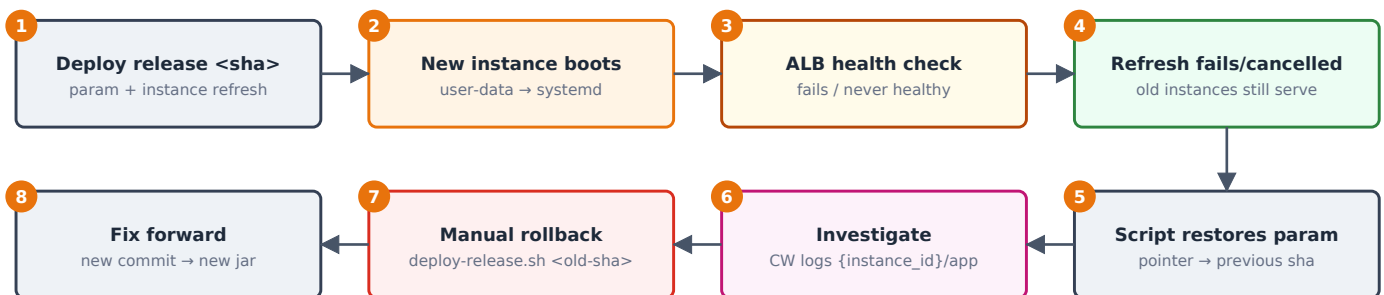
refresh_id=$(aws autoscaling start-instance-refresh \
  --auto-scaling-group-name "$ASG_NAME" \
  --strategy Rolling \
  --preferences '{"MinHealthyPercentage":100,"MaxHealthyPercentage":200,"InstanceWarmup":300}' \
  --region "$AWS_REGION" --query InstanceRefreshId --output text)
echo "Instance refresh $refresh_id started"

while true; do
  read -r status pct < <(aws autoscaling describe-instance-refreshes \
    --auto-scaling-group-name "$ASG_NAME" --instance-refresh-ids "$refresh_id" \
    --region "$AWS_REGION" \
    --query 'InstanceRefreshes[0].(Status,PercentageComplete)' --output text)
  echo "$(date -u +%H:%M:%S) status=$status complete=${pct}%"
  case "$status" in
    Successful) echo "Release $release is live"; exit 0 ;;
    Failed|Cancelled|RollbackSuccessful|RollbackFailed)
      echo "Refresh ended with $status. Restoring pointer to $previous" >&2
      aws ssm put-parameter --name "$RELEASE_PARAM" --value "$previous" --type String \
        --overwrite --region "$AWS_REGION" >/dev/null
      exit 1 ;;
  esac
  sleep 30
done

```

| Setting               | Value | Effect                                                            |
|-----------------------|-------|-------------------------------------------------------------------|
| MinHealthyPercentage  | 100   | Never drop below desired capacity: launch first, terminate after  |
| MaxHealthyPercentage  | 200   | Allows replacing all instances in parallel batches (faster)       |
| InstanceWarmup        | 300 s | Time before a new instance counts as healthy for refresh progress |
| ASG health_check_type | ELB   | Failing ALB checks → instance replaced                            |

## 3.2 Rolling back a failed release



If new instances never become healthy, the refresh stops and old instances keep serving (100% minimum healthy). The script restores the pointer so scale-out events launch the old release.

### rollback

```

# list releases still in the 90-day window
aws s3 ls s3://ai-lab-prod-artifacts-111122223333/api/
# stop a refresh that is still running
aws autoscaling cancel-instance-refresh --auto-scaling-group-name ai-lab-prod-api-asg
# roll back = redeploy the previous SHA (no upload). Or: Actions → deploy-ec2 → Run workflow → release=<sha>
AWS_REGION=us-east-1 ARTIFACT_BUCKET=ai-lab-prod-artifacts-111122223333 \
ASG_NAME=ai-lab-prod-api-asg RELEASE_PARAM=ai-lab/prod/api/release \
scripts/deploy-release.sh <previous-sha>

```

**Note** OS patching follows the same path: a new Amazon Linux 2023 AMI changes `data.aws_ssm_parameter.al2023_ami`, the next `terraform apply` creates a launch-template version, and the `instance_refresh` block (with `auto_rollback = true`) replaces instances. Schedule this regularly (e.g. weekly plan in CI).

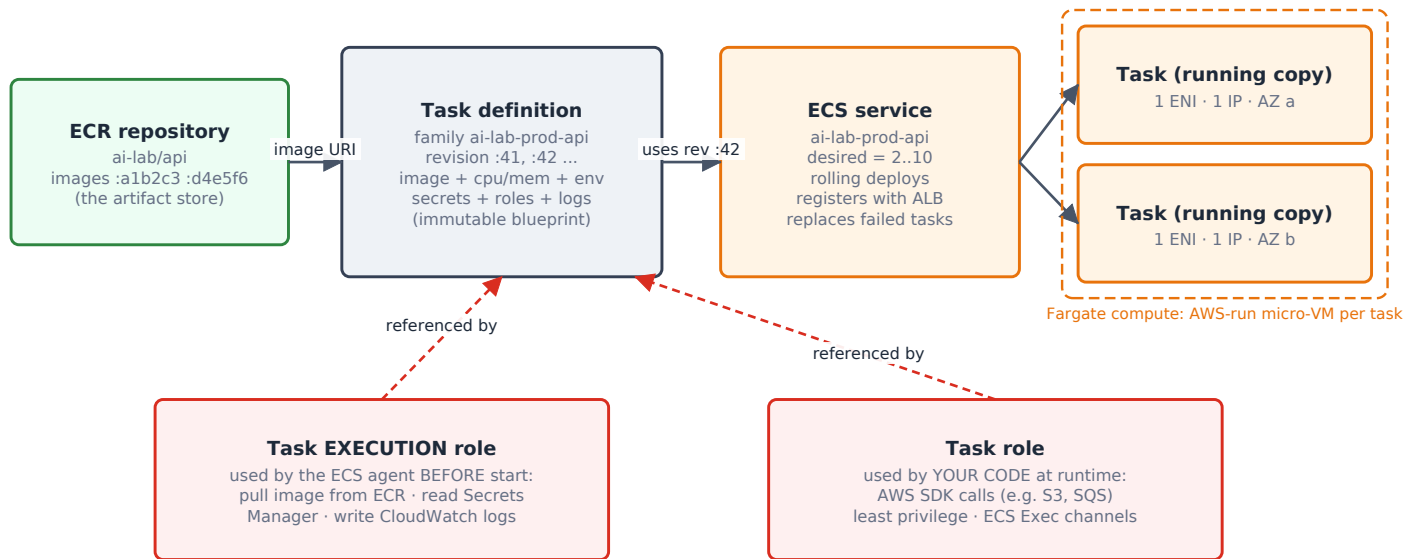
**Secret rotation** RDS-managed master secrets rotate automatically (7-day default schedule, configurable). Existing DB connections keep working, but connections opened after a rotation need the new password. On EC2 the fetcher re-reads the secret on every service restart, so schedule a rolling restart/refresh after rotation, lengthen the rotation schedule, or use a dedicated application DB user. Same consideration for ECS (`--force-new-deployment`).

## PART 4

## ECS Fargate deployment walkthrough

Commands run from `infrastructure/aws/ecs/` unless stated. Steps 1-2 (state bucket, app secret) are identical to Part 3.

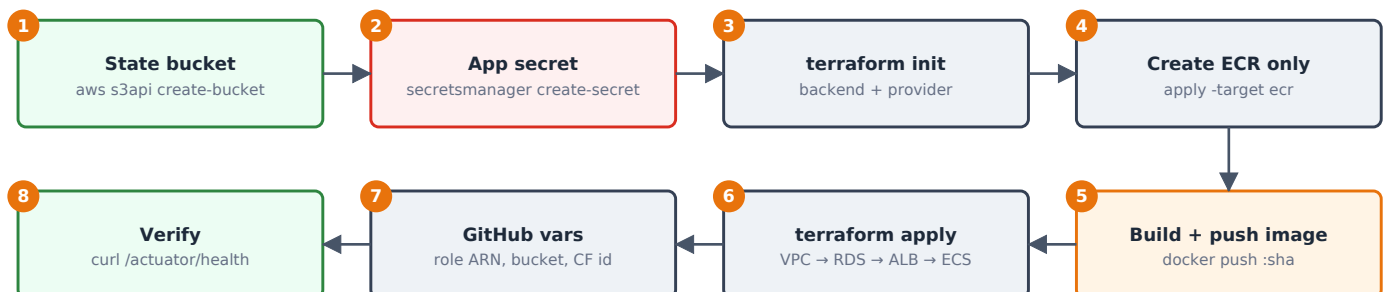
## 4.1 The ECS vocabulary in one picture



Cluster `ai-lab-prod` = namespace for services; with Fargate it contains no servers you manage.

Image (ECR) → blueprint (task definition revision) → controller (service) → running copies (tasks) on Fargate compute. Two different roles are attached to the blueprint.

| Concept             | What it is                                                                                                         | In this guide                                                                         |
|---------------------|--------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| ECR repository      | Private registry storing image versions.                                                                           | <code>ai-lab/api</code> , tag = git SHA, immutable                                    |
| Task definition     | Versioned JSON blueprint (image, CPU/memory, env, secrets, roles, logs, health check). Each change = new revision. | family <code>ai-lab-prod-api</code> , revisions <code>:1</code> , <code>:2</code> ... |
| Task                | One running instance of a task definition (one ENI/IP in <code>awsvpc</code> ).                                    | 2-10 tasks                                                                            |
| Service             | Keeps the desired number of tasks running, replaces failed ones, rolls deployments, registers tasks with the ALB.  | <code>ai-lab-prod-api</code>                                                          |
| Cluster             | Logical grouping of services/tasks. With Fargate it has no servers.                                                | <code>ai-lab-prod</code>                                                              |
| Fargate             | AWS-managed compute that runs each task in an isolated micro-VM.                                                   | <code>launch_type = "FARGATE"</code>                                                  |
| Task execution role | Used by the ECS agent to pull images, fetch <code>secrets</code> , create log streams.                             | <code>ai-lab-prod-api-execution-role</code>                                           |
| Task role           | Credentials your code gets via the AWS SDK at runtime.                                                             | <code>ai-lab-prod-api-task-role</code>                                                |



First-time order. ECR must exist and hold an image before the service starts, otherwise the first tasks fail with `CannotPullContainerError` and the circuit breaker trips.

## 1 Write the Dockerfile and build locally

|                     |                                                                                                               |
|---------------------|---------------------------------------------------------------------------------------------------------------|
| <b>File</b>         | <code>services/ai-engineering-api/Dockerfile</code> , <code>.dockerignore</code>                              |
| <b>AWS resource</b> | none (local image)                                                                                            |
| <b>Expected</b>     | <code>naming to docker.io/ai-lab/api:local</code> done; the container answers on 8080 when given DB settings. |

### Dockerfile

```
# syntax=docker/dockerfile:1
# Dockerfile - packages the backend as an OCI image for ECS Fargate.
# Build context: services/ai-engineering-api   Output: <ecr-url>/ai-lab/api:<git-sha>
# Production tip: pin both base images by digest (image@sha256:...) for reproducible builds.

# ---- Stage 1: build the jar (tests already ran in CI before this step) -----
FROM eclipse-temurin:21-jdk AS build
WORKDIR /workspace
COPY .mvn/ .mvn/
COPY mvnw pom.xml ./
RUN chmod +x mvnw && ./mvnw -B -q dependency:go-offline
COPY src/ src/
RUN ./mvnw -B -q package -DskipTests && cp target/*.jar /workspace/app.jar

# ---- Stage 2: minimal runtime -----
FROM eclipse-temurin:21-jre
# curl is used only by the ECS container health check.
RUN apt-get update \
  && apt-get install -y --no-install-recommends curl \
  && rm -rf /var/lib/apt/lists/* \
  && groupadd --system app \
  && useradd --system --gid app --home-dir /app --create-home app
WORKDIR /app
COPY --from=build --chown=app:app /workspace/app.jar /app/app.jar
USER app
EXPOSE 8080
# Size the heap from the TASK memory limit, not the host; exit (and get replaced) on OOM.
ENV JAVA_TOOL_OPTIONS="-XX:MaxRAMPercentage=75 -XX:+ExitOnOutOfMemoryError"
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

### shell

```
cd ../../../../services/ai-engineering-api
docker build -t ai-lab/api:local .
# on Apple Silicon build for the Fargate architecture in the task definition:
docker buildx build --platform linux/amd64 -t ai-lab/api:local .
```

| If it fails: symptom                                 | Likely cause → how to diagnose / fix                                                                         |
|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <code>./mvnw</code> : <code>Permission denied</code> | Handled by <code>chmod +x mvnw</code> in the Dockerfile; check line endings ( <code>.gitattributes</code> ). |
| Task later fails <code>exec format error</code>      | Image built for arm64 but task is <code>X86_64</code> (or vice versa).                                       |

## 2 Initialise Terraform and create only the ECR repository

|                     |                                                           |
|---------------------|-----------------------------------------------------------|
| <b>File</b>         | <code>versions.tf</code> , <code>ecr.tf</code>            |
| <b>AWS resource</b> | ECR repository <code>ai-lab/api</code> + lifecycle policy |
| <b>Expected</b>     | <code>Apply complete! Resources: 2 added.</code>          |

### ecr.tf excerpt

```
resource "aws_ecr_repository" "api" {
  name                = "${var.project}/api" # ai-lab/api
  image_tag_mutability = "IMMUTABLE"

  image_scanning_configuration {
    scan_on_push = true
  }

  encryption_configuration {
    encryption_type = "AES256"
  }
}
```

### shell

```
cp terraform.tfvars.example terraform.tfvars # edit values
terraform init
terraform apply -target=aws_ecr_repository.api -target=aws_ecr_lifecycle_policy.api
```

| If it fails: symptom                          | Likely cause → how to diagnose / fix                                         |
|-----------------------------------------------|------------------------------------------------------------------------------|
| <code>RepositoryAlreadyExistsException</code> | Import it: <code>terraform import aws_ecr_repository.api ai-lab/api</code> . |

### 3 Push the first image

|                     |                                                                                                                                 |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <b>AWS resource</b> | Image <code>111122223333.dkr.ecr.us-east-1.amazonaws.com/ai-lab/api:&lt;sha&gt;</code>                                          |
| <b>Expected</b>     | Login Succeeded, then <code>&lt;sha&gt;: digest: sha256:... size: ...</code> . Put that SHA in <code>initial_image_tag</code> . |

shell

```
REGISTRY=111122223333.dkr.ecr.us-east-1.amazonaws.com
SHA=$(git rev-parse HEAD)
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin "$REGISTRY"
docker tag ai-lab/api:local "$REGISTRY/ai-lab/api:$SHA"
docker push "$REGISTRY/ai-lab/api:$SHA"
aws ecr describe-image-scan-findings --repository-name ai-lab/api --image-id imageTag=$SHA \
  --query 'imageScanFindings.findingSeverityCounts'
```

| If it fails: symptom           | Likely cause → how to diagnose / fix                                        |
|--------------------------------|-----------------------------------------------------------------------------|
| no basic auth credentials      | Login expired (tokens last 12 h) → repeat <code>get-login-password</code> . |
| tag invalid ... already exists | Immutable tags: each build needs a new tag (use the commit SHA).            |

### 4 Plan and apply everything else

|                     |                                                                                                                                  |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>File</b>         | all <code>*.tf</code>                                                                                                            |
| <b>AWS resource</b> | VPC ... RDS, ALB, ACM, CloudFront, IAM roles, cluster, log group, task definition :1, service, scaling target + policies, alarms |
| <b>Expected</b>     | Apply complete!; the service is created and starts 2 tasks. Roughly 15-40 min (RDS, CloudFront).                                 |

shell

```
terraform plan -out=tfplan
terraform apply tfplan
```

| If it fails: symptom                                                                      | Likely cause → how to diagnose / fix                                                                      |
|-------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| InvalidParameterException: ... target group ... does not have an associated load balancer | Listener not yet created; <code>depends_on = [aws_lb_listener.https]</code> in the service prevents this. |
| Apply finishes but tasks keep stopping                                                    | Diagnose in step 6 (stopped reasons).                                                                     |

### 5 Task definition, service and target group: what was registered

|                     |                                                                                       |
|---------------------|---------------------------------------------------------------------------------------|
| <b>File</b>         | <code>ecs-task-definition.tf</code> , <code>ecs-service.tf</code>                     |
| <b>AWS resource</b> | Task definition <code>ai-lab-prod-api:1</code> , service <code>ai-lab-prod-api</code> |
| <b>Expected</b>     | <code>rolloutState = COMPLETED</code> , <code>running = desired</code> .              |

`ecs-service.tf` excerpt

```
resource "aws_ecs_service" "api" {
  name         = "${local.name}-api"
  cluster      = aws_ecs_cluster.main.id
  task_definition = aws_ecs_task_definition.api.arn
  desired_count = var.service_desired_count
  launch_type  = "FARGATE"
  platform_version = "LATEST"
  enable_execute_command = true # OPTIONAL: ECS Exec for debugging
  propagate_tags = "SERVICE"

  # Ignore ALB health checks for this long after a task starts (JVM warm-up).
  health_check_grace_period_seconds = 120

  # Rolling update: start new tasks first (up to 200%), never drop below 100% healthy.
  deployment_minimum_healthy_percent = 100
  deployment_maximum_percent        = 200

  deployment_controller {
    type = "ECS"
  }

  # Roll back automatically when new tasks keep failing to start or to pass health checks.
  deployment_circuit_breaker {
    enable   = true
    rollback = true
  }

  # Also roll back when this alarm fires during the deployment.
  alarms {
    enable      = true
    rollback    = true
    alarm_names = [aws_cloudwatch_metric_alarm.api_5xx.alarm_name]
  }

  network_configuration {
    subnets         = aws_subnet.private[*].id
    security_groups   = [aws_security_group.app.id]
    assign_public_ip = false
  }

  load_balancer {
    target_group_arn = aws_lb_target_group.api.arn
  }
}
```

**ecs-service.tf** excerpt

```

    container_name = "api"
    container_port = var.api_port
  }

  depends_on = [aws_lb_listener.https]

  lifecycle {
    # task_definition: CI registers new revisions and points the service at them.
    # desired_count : Service Auto Scaling owns it after creation.
    ignore_changes = [task_definition, desired_count]
  }
}

```

**shell**

```

aws ecs describe-services --cluster ai-lab-prod --services ai-lab-prod-api \
  --query 'services[0].{status:status,desired:desiredCount,running:runningCount,
    taskDef:taskDefinition,rollout:deployments[0].rolloutState}'
aws ecs describe-services --cluster ai-lab-prod --services ai-lab-prod-api \
  --query 'services[0].events[:5].message'

```

**6 Diagnose tasks that do not stay up**

|                     |                                                                                   |
|---------------------|-----------------------------------------------------------------------------------|
| <b>AWS resource</b> | Stopped tasks (kept visible for a short time after stopping)                      |
| <b>Expected</b>     | <code>stoppedReason</code> and container <code>reason</code> explain the failure. |

**shell**

```

aws ecs list-tasks --cluster ai-lab-prod --service-name ai-lab-prod-api --desired-status STOPPED
aws ecs describe-tasks --cluster ai-lab-prod --tasks <task-arn> \
  --query 'tasks[0].{stopped:stoppedReason,containers:[].{name,lastStatus,exitCode,reason}}'
aws logs tail /ecs/ai-lab-prod-api --since 30m

```

| If it fails: symptom                                             | Likely cause → how to diagnose / fix                                                                                                 |
|------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <code>CannotPullContainerError</code>                            | No route to ECR (NAT/endpoints), wrong image tag, or execution role missing ECR permissions.                                         |
| <code>ResourceInitializationError: unable to pull secrets</code> | Execution role lacks <code>GetSecretValue</code> on that ARN, JSON key missing in the secret, or no network path to Secrets Manager. |
| <code>Essential container in task exited</code>                  | App crashed → read the log stream <code>api/api/&lt;task-id&gt;</code> .                                                             |
| <code>Task failed ELB health checks</code>                       | Wrong path/port, SG, or start-up slower than <code>health_check_grace_period_seconds</code> .                                        |
| <code>OutOfMemoryError</code> / exit 137                         | Raise <code>task_memory</code> , keep <code>MaxRAMPercentage=75</code> .                                                             |

**7 Check the ALB target group**

|                     |                                                                                |
|---------------------|--------------------------------------------------------------------------------|
| <b>AWS resource</b> | Target group <code>ai-lab-prod-api-tg</code> ( <code>target_type = ip</code> ) |
| <b>Expected</b>     | Two private IPs (one per AZ), state <code>healthy</code> .                     |

**shell**

```

TG=$(aws elbv2 describe-target-groups --names ai-lab-prod-api-tg --query 'TargetGroups[0].TargetGroupArn' --output text)
aws elbv2 describe-target-health --target-group-arn "$TG" \
  --query 'TargetHealthDescriptions[0].[Target.Id,Target.AvailabilityZone,TargetHealth.State]' --output table

```

**8 Verify autoscaling**

|                     |                                                                                                                        |
|---------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>File</b>         | <code>autoscaling.tf</code>                                                                                            |
| <b>AWS resource</b> | Scalable target <code>service/ai-lab-prod/ai-lab-prod-api</code> , 2 target-tracking policies (+ their managed alarms) |
| <b>Expected</b>     | Min 2 / max 10; two policies listed.                                                                                   |

**shell**

```

aws application-autoscaling describe-scalable-targets --service-namespace ecs \
  --resource-ids service/ai-lab-prod/ai-lab-prod-api
aws application-autoscaling describe-scaling-policies --service-namespace ecs \
  --resource-id service/ai-lab-prod/ai-lab-prod-api --query 'ScalingPolicies[0].PolicyName'

```

**9 HTTPS, DNS and GitHub wiring**

|                     |                                                                                                                                                                                             |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>AWS resource</b> | as in Part 3 steps 9 and 11                                                                                                                                                                 |
| <b>Expected</b>     | <code>https://api.example.com/actuator/health</code> is UP; GitHub variables <code>AWS_DEPLOY_ROLE_ARN</code> , <code>FRONTEND_BUCKET</code> , <code>CLOUDFRONT_DISTRIBUTION_ID</code> set. |

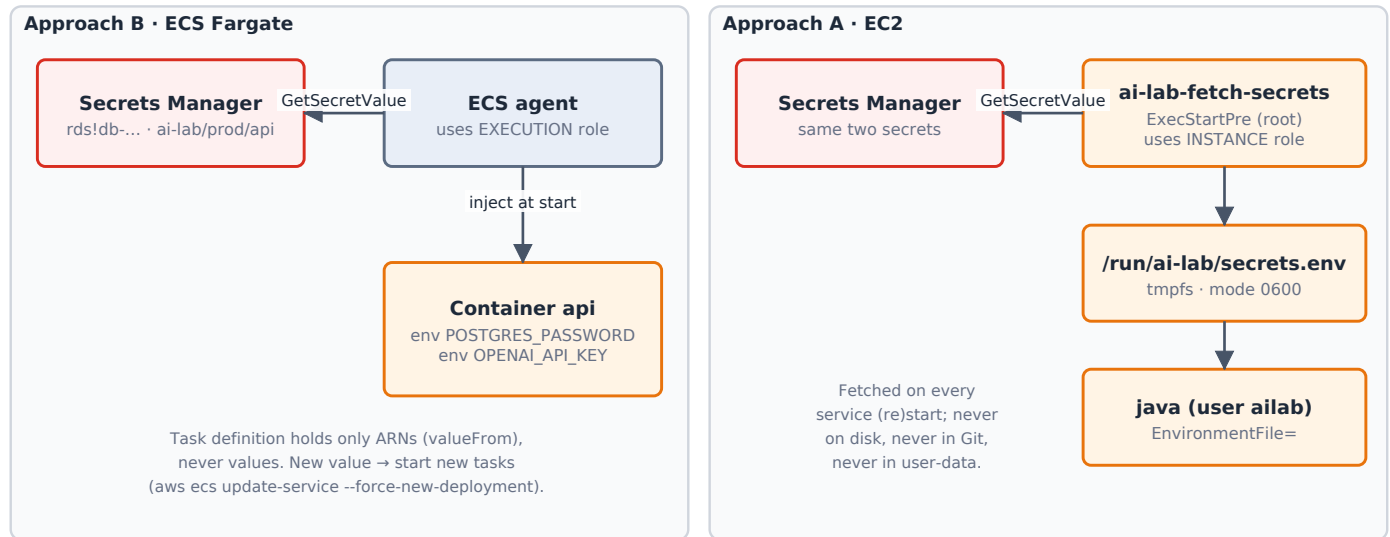
**shell**

```

curl -sS https://api.example.com/actuator/health
for v in AWS_DEPLOY_ROLE_ARN:github_deploy_role_arn FRONTEND_BUCKET:frontend_bucket \
  CLOUDFRONT_DISTRIBUTION_ID:cloudfront_distribution_id; do
  gh variable set "${v%:*}" --env production --body "${terraform output -raw ${v##*:}}"
done

```

## 4.2 How tasks receive configuration and secrets



Left: the ECS agent resolves `secrets[].valueFrom` with the execution role at task start and injects plain environment variables; the task definition only contains ARNs. Right: the EC2 equivalent.

### ecs-task-definition.tf complete resource

```
resource "aws_ecs_task_definition" "api" {
  family           = "${local.name}-api"
  requires_compatibilities = ["FARGATE"]
  network_mode     = "awsvpc"
  cpu              = var.task_cpu
  memory           = var.task_memory
  execution_role_arn = aws_iam_role.ecs_execution.arn
  task_role_arn     = aws_iam_role.ecs_task.arn

  runtime_platform {
    operating_system_family = "LINUX"
    cpu_architecture       = "X86_64" # use ARM64 only if the image is built for arm64
  }

  container_definitions = jsonencode([
    {
      name       = "api"
      image      = "${aws_ecr_repository.api.repository_url}:${var.initial_image_tag}"
      essential = true

      portMappings = [{ containerPort = var.api_port, protocol = "tcp" }]

      # Plain, non-sensitive configuration.
      environment = [
        { name = "SERVER_PORT", value = tostring(var.api_port) },
        { name = "POSTGRES_HOST", value = aws_db_instance.main.address },
        { name = "POSTGRES_PORT", value = "5432" },
        { name = "POSTGRES_DB", value = var.db_name },
      ]

      # Resolved by the ECS agent at task start using the EXECUTION role.
      # Format: <secret-arn>:<json-key>:<version-stage>:<version-id>
      secrets = [
        { name = "POSTGRES_USER", valueFrom = "${local.db_secret_arn}:username:" },
        { name = "POSTGRES_PASSWORD", valueFrom = "${local.db_secret_arn}:password:" },
        { name = "OPENAI_API_KEY", valueFrom = "${data.aws_secretsmanager_secret.app.arn}:OPENAI_API_KEY:" },
      ]

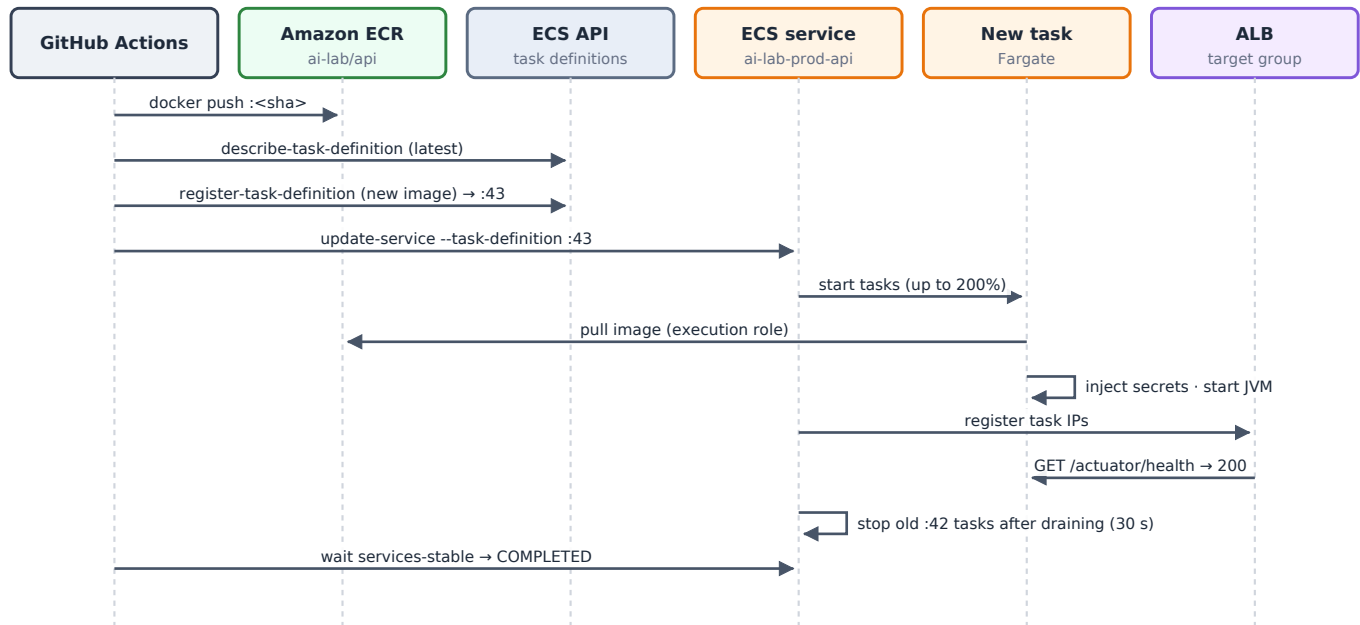
      # Container-level health check (runs inside the container; image needs curl).
      healthCheck = {
        command = ["CMD-SHELL", "curl -fsS http://localhost:${var.api_port}${var.health_check_path} || exit 1"]
        interval = 30
        timeout = 5
        retries = 3
        startPeriod = 120 # JVM start-up grace; failures are ignored during this window
      }

      logConfiguration = {
        logDriver = "awslogs"
        options = {
          awslogs-group = aws_cloudwatch_log_group.api.name
          awslogs-region = var.aws_region
          awslogs-stream-prefix = "api"
        }
      }

      stopTimeout = 30 # seconds between SIGTERM and SIGKILL (graceful shutdown)
    }
  ])
}
```

| Field                                | Meaning                                                                                                                        |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <code>environment</code>             | Plain values, visible in the console/API. Never secrets.                                                                       |
| <code>secrets[].valueFrom</code>     | <code>&lt;secret-arn&gt;:&lt;json-key&gt;:&lt;version-stage&gt;:&lt;version-id&gt;</code> ; empty trailing parts = AWSCURRENT. |
| <code>healthCheck.startPeriod</code> | Grace period during which failed container checks do not count.                                                                |
| <code>logConfiguration</code>        | <code>awslogs</code> driver → stream <code>api/api/&lt;task-id&gt;</code> in <code>/ecs/ai-lab-prod-api</code> .               |
| <code>cpu / memory</code>            | Task-level size; must be a valid Fargate combination (e.g. 1024/2048).                                                         |

## 4.3 Deploying new image versions



CI never edits Terraform. It clones the latest revision, swaps the image and points the service at the new revision. Terraform ignores `task_definition` on the service, so it will not revert this.

### manual deploy

```
# manual equivalent of the CI job (requires jq)
IMAGE=111122223333.dkr.ecr.us-east-1.amazonaws.com/ai-lab/api:$SHA
aws ecs describe-task-definition --task-definition ai-lab-prod-api --query taskDefinition > td.json
jq --arg IMG "$IMAGE" '.containerDefinitions[0].image = $IMG' \
  | del(.taskDefinitionArn,.revision,.status,.requiresAttributes,.compatibilities,.registeredAt,.registeredBy)' \
  td.json > td-new.json
REV=$(aws ecs register-task-definition --cli-input-json file://td-new.json --query taskDefinition.revision --output text)
aws ecs update-service --cluster ai-lab-prod --service ai-lab-prod-api --task-definition ai-lab-prod-api:$REV
aws ecs wait services-stable --cluster ai-lab-prod --services ai-lab-prod-api
```

**Caution** A Terraform change to the task definition (e.g. a new environment variable) registers a new revision but does **not** deploy it (`ignore_changes`). Roll it out by running the `deploy-ecs` workflow manually (`workflow_dispatch`): it copies the latest revision, which now contains your change, and sets the current image. Never run `update-service --task-definition ai-lab-prod-api` without a revision number: the latest revision created by Terraform carries `initial_image_tag`, an old image.

## 4.4 Rolling deployments and automatic rollback

Rolling update with `minimumHealthyPercent=100`, `maximumPercent=200`, `desired=2`



### Circuit breaker (enable + rollback)

Counts tasks of the NEW revision that fail to reach RUNNING or fail ALB/container health checks. Threshold = 50% of desired, bounded 3..200 → with desired 2 → 3 failures. Then ECS redeploys the last COMPLETED revision.

### CloudWatch alarm rollback (alarms block)

If `ai-lab-prod-api-5xx` goes to ALARM during the deployment (while it is `IN_PROGRESS`), ECS marks the deployment FAILED and rolls back. Catches 'healthy but broken' releases the health check misses.

With 100/200 the service starts a full new set before stopping the old one, so capacity never drops. Old tasks are drained for `deregistration_delay` (30 s) before they stop.

| ECS rolls back automatically when...                                                                                          | ECS does NOT detect...                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| New tasks repeatedly fail to reach RUNNING (image pull, secrets, crash on start) and the failure count reaches the threshold. | A release that passes health checks but returns wrong data (add functional checks/alarms). |
| New tasks repeatedly fail ALB target health checks or the container <code>healthCheck</code> .                                | Problems that start after the deployment reached COMPLETED.                                |
| A CloudWatch alarm listed in the service <code>alarms</code> block enters ALARM during the deployment.                        | Database migrations that already ran: rolling back the code does not roll back the schema. |

### rollback

```
# manual rollback to a known-good revision
aws ecs list-task-definitions --family-prefix ai-lab-prod-api --sort DESC --max-items 5
aws ecs update-service --cluster ai-lab-prod --service ai-lab-prod-api --task-definition ai-lab-prod-api:41
aws ecs wait services-stable --cluster ai-lab-prod --services ai-lab-prod-api
# force fresh tasks with the same revision (e.g. after a secret rotation)
aws ecs update-service --cluster ai-lab-prod --service ai-lab-prod-api --force-new-deployment
```

## 4.5 Shell access and DB port forwarding without hosts

### ECS Exec (optional)

```
TASK=$(aws ecs list-tasks --cluster ai-lab-prod --service-name ai-lab-prod-api --query 'taskArns[0]' --output text)
aws ecs execute-command --cluster ai-lab-prod --task "$TASK" --container api --interactive --command "/bin/sh"
# port-forward to RDS through the task (ECS Exec target format: ecs:<cluster>_<task-id>_<runtime-id>)
RID=$(aws ecs describe-tasks --cluster ai-lab-prod --tasks "$TASK" \
  --query 'tasks[0].containers[0].runtimeId' --output text)
aws ssm start-session --target "ecs:ai-lab-prod_${TASK##*/}_${RID}" \
  --document-name AWS-StartPortForwardingSessionToRemoteHost \
  --parameters '{"host":["ai-lab-prod-db.abcdefghijkl.us-east-1.rds.amazonaws.com"],\
    "portNumber":["5432"],"localPortNumber":["15432"]}'
```

**Note** ECS Exec needs `enable_execute_command = true` (set), the `ssmmessages` permissions on the task role (set) and outbound 443. It only applies to tasks started as enabled. Disable it if you do not need it.

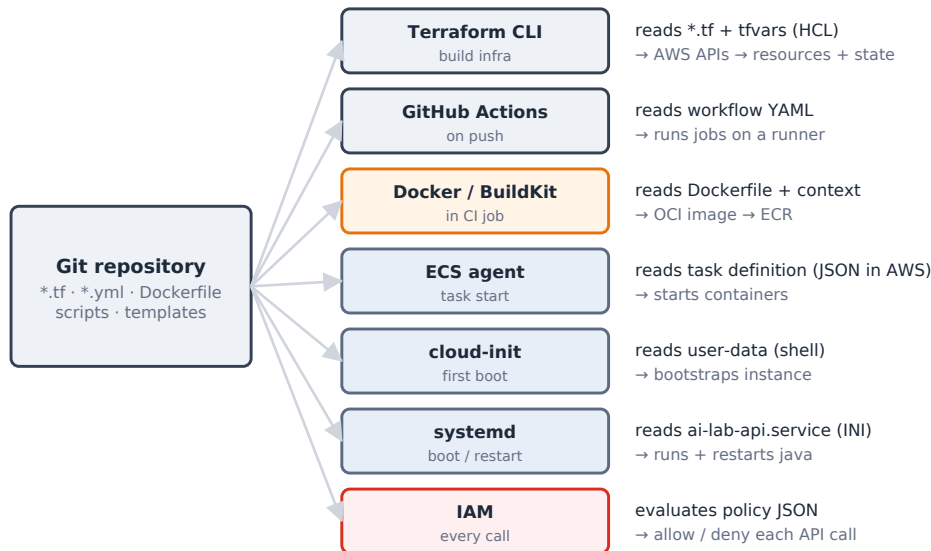
## PART 5

## YAML, JSON, HCL and shell: file-by-file

Complete files are in the Appendix. Here: who reads each format, in which order, and the fields that matter.

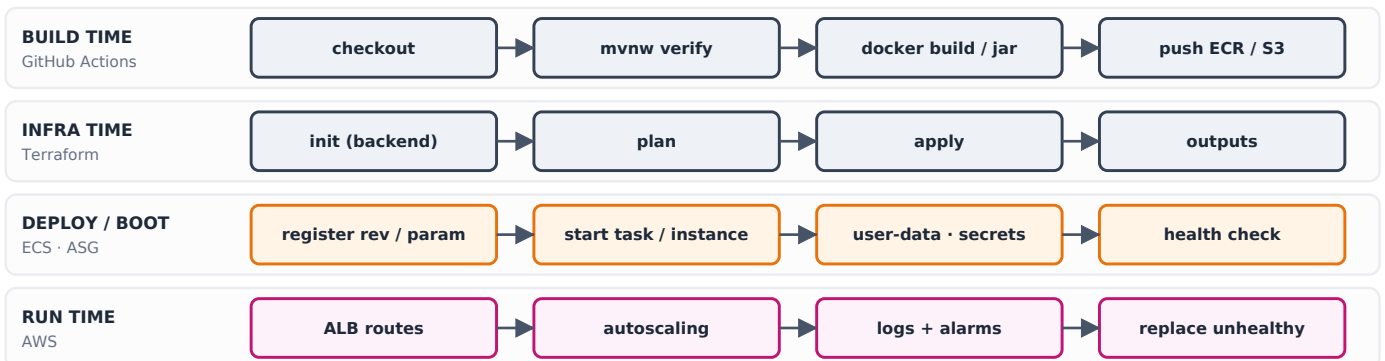
**Formats Terraform HCL is not YAML.** HCL uses **blocks** `{ }`, **key = value** and expressions such as `aws_lb.main.arn`; YAML uses indentation and **key: value**. Most AWS resource configuration in this guide lives in HCL, not in YAML; the only YAML files are the GitHub Actions workflows. The task definition is JSON stored inside ECS; systemd units are INI; user-data is a shell script.

## 5.1 Who reads what



Each format has exactly one reader. Knowing the reader tells you when a change takes effect: Terraform on apply, workflows on the next push, user-data on the next new instance, the task definition on the next task start.

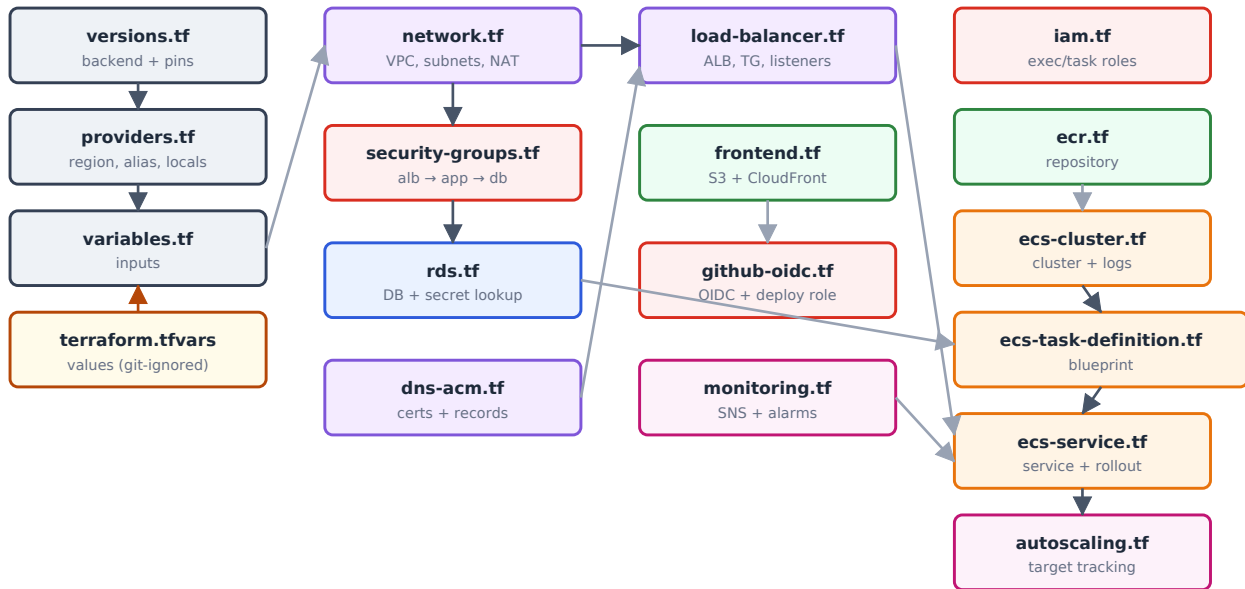
## 5.2 Execution order



Order across lanes: Terraform first (once), then every release = BUILD → DEPLOY → RUN. Infra changes re-enter at INFRA TIME.

Infrastructure is created once and changed rarely; releases loop through build → deploy → run.

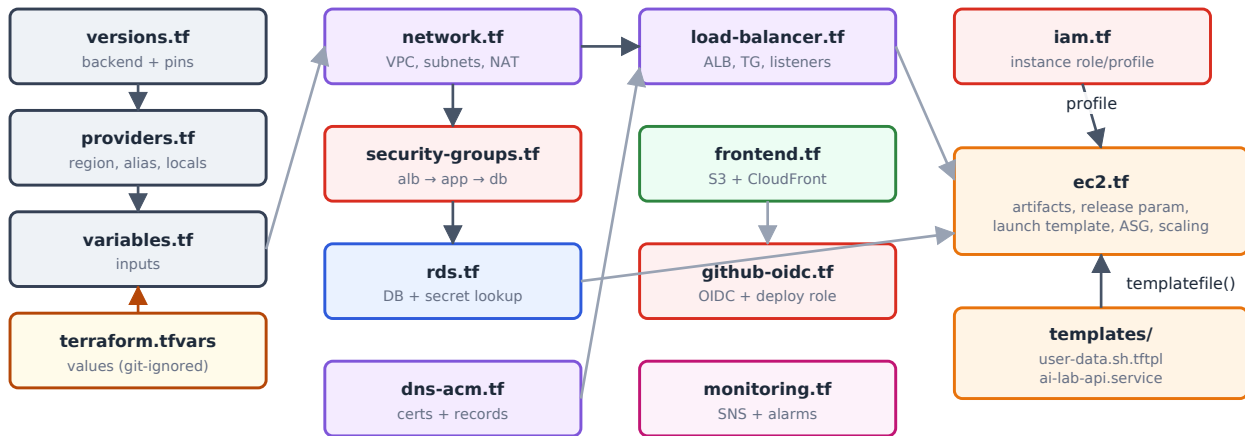
## 5.3 Terraform HCL



Terraform reads ALL \*.tf files in a directory as ONE configuration; file names are only for humans.

Dark arrows: dependency order. Grey arrows: cross-file references (cert ARN, target group, DB address + secret ARN, alarm).

ECS stack: references between files. Terraform builds the dependency graph from these references, not from file order.



Terraform reads ALL \*.tf files in a directory as ONE configuration; file names are only for humans.

Dark arrows: dependency order. Grey arrows: cross-file references (cert ARN, target group, DB address + secret ARN, alarm).

EC2 stack: same shared core; **ec2.tf** renders the templates into the launch template.

| Block / syntax                         | Example from these files                                  | Meaning                                                                     |
|----------------------------------------|-----------------------------------------------------------|-----------------------------------------------------------------------------|
| <code>terraform { backend ... }</code> | <code>use_lockfile = true</code>                          | Where state lives; S3 lock file prevents concurrent applies                 |
| <code>provider + alias</code>          | <code>provider = aws.us_east_1</code>                     | Second region just for the CloudFront certificate                           |
| <code>variable / validation</code>     | <code>az_count</code> must be 2-3                         | Typed inputs; fails fast on bad values                                      |
| <code>locals</code>                    | <code>name = "\${var.project}-\${var.environment}"</code> | Computed names reused everywhere (consistency)                              |
| <code>data</code>                      | <code>data.aws_route53_zone.main</code>                   | Look up something that already exists (never created/destroyed)             |
| <code>resource</code>                  | <code>aws_lb.main</code>                                  | Something Terraform creates, updates and destroys                           |
| <code>reference</code>                 | <code>aws_lb_target_group.api.arn</code>                  | Creates an implicit dependency (ordering)                                   |
| <code>count / for_each</code>          | <code>aws_subnet.private[count.index]</code>              | One block → many resources                                                  |
| <code>lifecycle.ignore_changes</code>  | <code>[task_definition, desired_count]</code>             | Hand ownership of a field to CI/autoscaling                                 |
| <code>templatefile()</code>            | user-data with <code>\${db_host}</code>                   | Render a file with Terraform values; <code>`\${x}`</code> escapes a literal |
| <code>output</code>                    | <code>terraform output -raw asg_name</code>               | Values handed to CI and humans                                              |

## 5.4 GitHub Actions YAML

**deploy-ecs.yml** header (full file in Appendix)

```
# .github/workflows/deploy-ecs.yml - test -> build image -> push to ECR -> roll ECS service
# -> publish React to S3 -> invalidate CloudFront. AWS access via OIDC (no stored keys).
name: deploy-ecs

on:
  push:
    branches: [main]
  workflow_dispatch: {} # manual re-deploy, e.g. after a Terraform task-definition change

permissions:
  id-token: write # required to request the GitHub OIDC token
  contents: read

concurrency:
  group: deploy-prod # never run two production deploys at once
  cancel-in-progress: false

env:
  AWS_REGION: us-east-1
  ECR_REPOSITORY: ai-lab/api
  ECS_CLUSTER: ai-lab-prod
  ECS_SERVICE: ai-lab-prod-api
  TASK_FAMILY: ai-lab-prod-api
  CONTAINER_NAME: api
```

| Key                                     | Why it matters                                                                                                                           |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <b>permissions: id-token: write</b>     | Lets the job request an OIDC token; without it AssumeRoleWithWebIdentity fails                                                           |
| <b>environment: production</b>          | Becomes the token <b>sub</b> ( <b>repo:OWNER/REPO:environment:production</b> ) that the IAM trust policy requires; can enforce reviewers |
| <b>concurrency.group</b>                | Serialises production deploys; <b>cancel-in-progress: false</b> never kills a half-done rollout                                          |
| <b>needs:</b>                           | Orders jobs: tests → API → web                                                                                                           |
| <b>uses: / with:</b>                    | Pinned major versions of official actions ( <b>aws-actions/*</b> ). Pin to commit SHAs for stricter supply-chain control                 |
| <b>vars.*</b>                           | Non-secret GitHub variables (role ARN, bucket names). No <b>secrets.*</b> are needed with OIDC                                           |
| <b>wait-for-service-stability: true</b> | The job fails if the deployment does not reach steady state (e.g. circuit-breaker rollback)                                              |

## 5.5 Dockerfile, ECS task definition JSON, IAM JSON

| Dockerfile line                                      | Purpose                                                                         |
|------------------------------------------------------|---------------------------------------------------------------------------------|
| <b>FROM ... AS build</b> / second <b>FROM</b>        | Multi-stage: build tools stay out of the runtime image                          |
| <b>COPY mvnw pom.xml</b> before <b>COPY src/</b>     | Dependency layer is cached between builds                                       |
| <b>USER app</b>                                      | Container runs as non-root                                                      |
| <b>ENV JAVA_TOOL_OPTIONS=-XX:MaxRAMPercentage=75</b> | Heap sized from the task memory limit; leaves room for metaspace/threads        |
| <b>EXPOSE 8080</b>                                   | Documentation only; the port is opened by <b>portMappings</b> + security groups |

**ecs-task-definition.json** what ECS stores (rendered from Terraform; reference only)

```
{
  "family": "ai-lab-prod-api",
  "requiresCompatibilities": ["FARGATE"],
  "networkMode": "awsvpc",
  "cpu": "1024",
  "memory": "2048",
  "runtimePlatform": { "operatingSystemFamily": "LINUX", "cpuArchitecture": "X86_64" },
  "executionRoleArn": "arn:aws:iam::11112223333:role/ai-lab-prod-api-execution-role",
  "taskRoleArn": "arn:aws:iam::11112223333:role/ai-lab-prod-api-task-role",
  "containerDefinitions": [
    {
      "name": "api",
      "image": "11112223333.dkr.ecr.us-east-1.amazonaws.com/ai-lab/api:0123abc",
      "essential": true,
      "portMappings": [{ "containerPort": 8080, "protocol": "tcp" }],
      "environment": [
        { "name": "SERVER_PORT", "value": "8080" },
        { "name": "POSTGRES_HOST", "value": "ai-lab-prod-db.abcdefghijk.us-east-1.rds.amazonaws.com" },
        { "name": "POSTGRES_PORT", "value": "5432" },
        { "name": "POSTGRES_DB", "value": "ai_lab" }
      ],
      "secrets": [
        { "name": "POSTGRES_USER", "valueFrom": "arn:aws:secretsmanager:us-east-1:11112223333:secret:rds!db-1a2b3c-AbCdEf:username::" },
        { "name": "POSTGRES_PASSWORD", "valueFrom": "arn:aws:secretsmanager:us-east-1:11112223333:secret:rds!db-1a2b3c-AbCdEf:password::" },
        { "name": "OPENAI_API_KEY", "valueFrom": "arn:aws:secretsmanager:us-east-1:11112223333:secret:ai-lab/prod/api-GhIjKl:OPENAI_API_KEY::" }
      ],
      "healthCheck": {
        "command": ["CMD-SHELL", "curl -fsS http://localhost:8080/actuator/health || exit 1"],
        "interval": 30, "timeout": 5, "retries": 3, "startPeriod": 120
      }
    }
  ]
}
```

**ecs-task-definition.json** what ECS stores (rendered from Terraform; reference only)

```

    "logConfiguration": {
      "logDriver": "awslogs",
      "options": {
        "awslogs-group": "/ecs/ai-lab-prod-api",
        "awslogs-region": "us-east-1",
        "awslogs-stream-prefix": "api"
      }
    },
    "stopTimeout": 30
  }
}

```

**IAM trust policy** (rendered from github-oidc.tf)

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Federated": "arn:aws:iam::111122223333:oidc-provider/token.actions.githubusercontent.com"
      },
      "Action": "sts:AssumeRoleWithWebIdentity",
      "Condition": {
        "StringEquals": {
          "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
          "token.actions.githubusercontent.com:sub": "repo:OWNER/ai-engineering-lab:environment:production"
        }
      }
    }
  ]
}

```

| IAM JSON element | Meaning                                                                                                           |
|------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Principal</b> | WHO may assume the role (a service such as <code>ecs-tasks.amazonaws.com</code> , or a federated OIDC provider)   |
| <b>Action</b>    | API operations allowed ( <code>ecr:PutImage</code> , <code>secretsmanager:GetSecretValue</code> ...)              |
| <b>Resource</b>  | Exact ARNs. * only where the API has no resource-level permissions (e.g. <code>ecr:GetAuthorizationToken</code> ) |
| <b>Condition</b> | Extra checks: OIDC <code>aud/sub</code> , <code>aws:SourceAccount</code> , <code>iam:PassedToService</code>       |

## 5.6 Shell, systemd, CloudWatch agent

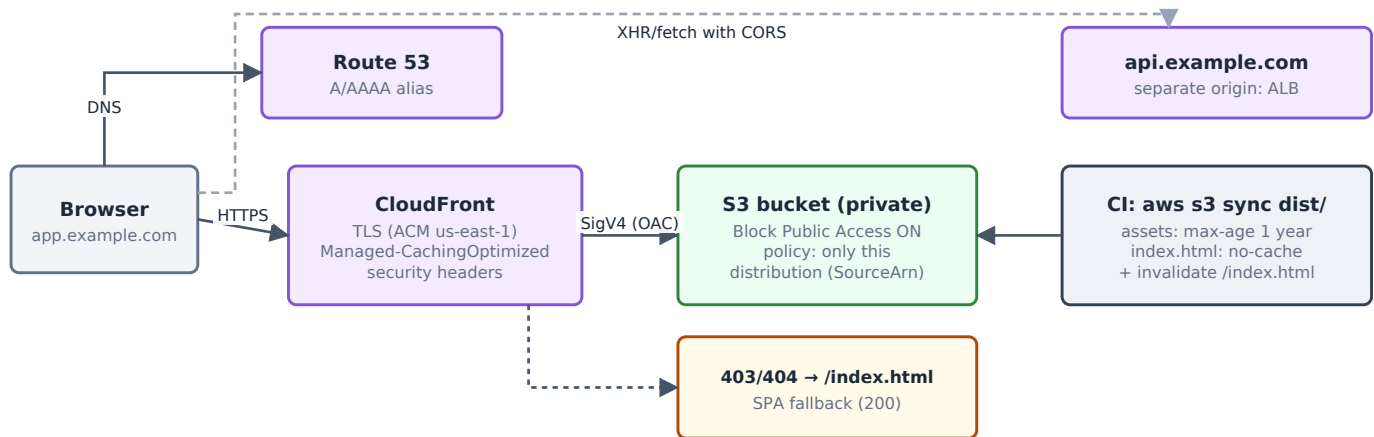
| File          | Key lines                                                                     | Why                                                                                  |
|---------------|-------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| user-data     | <code>set -euo pipefail; exec &gt; &gt;(tee -a /var/log/user-data.log)</code> | Stop on first error; keep a log you can read via SSM and CloudWatch                  |
| user-data     | <code>cat &gt; ... &lt;&lt;'SCRIPT'</code> (quoted)                           | No shell expansion while writing helper scripts                                      |
| systemd unit  | <code>ExecStartPre+=...fetch-secrets</code>                                   | <code>+</code> = run that one command as root; the app runs as <code>ailab</code>    |
| systemd unit  | <code>EnvironmentFile=-/run/ai-lab/secrets.env</code>                         | <code>-</code> = tolerate absence before the first fetch; <code>/run</code> is tmpfs |
| systemd unit  | <code>ProtectSystem=strict,ReadWritePaths=/opt/ai-lab</code>                  | Read-only OS for the app process                                                     |
| CW agent JSON | <code>log_stream_name: {instance_id}/app</code>                               | One stream per instance; easy to correlate with ASG activity                         |
| CW agent JSON | <code>append_dimensions + aggregation_dimensions</code>                       | Memory/disk metrics per ASG (EC2 has no built-in memory metric)                      |

**Note** AWS CLI commands are not configuration: anything you create with the CLI (state bucket, app secret) is outside Terraform. Keep that list short and documented (Part 10 checklist).

## PART 6

# React hosting: S3 + CloudFront

The React build is a folder of static files. It needs storage, a CDN and TLS, not a server.



Private bucket, CloudFront with Origin Access Control, custom domain with a us-east-1 certificate. The API is a separate origin ([api.example.com](http://api.example.com)), so the browser makes cross-origin calls.

**frontend.tf** distribution (bucket, OAC and policy in Appendix)

```

resource "aws_cloudfront_distribution" "frontend" {
  enabled             = true
  comment             = "${local.name} React SPA"
  aliases             = [local.app_fqdn]
  default_root_object = "index.html"
  is_ipv6_enabled     = true
  http_version       = "http2and3"
  price_class        = "PriceClass_100" # cheapest edge set; widen if users are global

  origin {
    origin_id           = "s3-frontend"
    domain_name         = aws_s3_bucket.frontend.bucket_regional_domain_name
    origin_access_control_id = aws_cloudfront_origin_access_control.frontend.id
  }

  default_cache_behavior {
    target_origin_id           = "s3-frontend"
    viewer_protocol_policy     = "redirect-to-https"
    allowed_methods            = ["GET", "HEAD"]
    cached_methods            = ["GET", "HEAD"]
    compress                   = true
    cache_policy_id            = data.aws_cloudfront_cache_policy.caching_optimized.id
    response_headers_policy_id = data.aws_cloudfront_response_headers_policy.security_headers.id
  }

  # SPA fallback: a deep link such as /orders/42 is not an S3 object. Without
  # s3:ListBucket S3 answers 403 (not 404), so map both to index.html with 200.
  custom_error_response {
    error_code           = 403
    response_code        = 200
    response_page_path   = "/index.html"
    error_caching_min_ttl = 10
  }

  custom_error_response {
    error_code           = 404
    response_code        = 200
    response_page_path   = "/index.html"
    error_caching_min_ttl = 10
  }

  restrictions {
    geo_restriction {
      restriction_type = "none"
    }
  }

  viewer_certificate {
    acm_certificate_arn       = aws_acm_certificate_validation.app.certificate_arn
    ssl_support_method        = "sni-only"
    minimum_protocol_version = "TLSv1.2_2021"
  }
}

```

| Topic         | How it is handled                                                                                                                                                                            |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Upload        | <code>aws s3 sync dist/ s3://&lt;bucket&gt; --delete --exclude index.html --cache-control public,max-age=31536000,immutable, then index.html with no-cache.</code>                           |
| Bucket access | Block Public Access on, ACLs disabled, policy allows <code>s3:GetObject</code> only for principal <code>cloudfront.amazonaws.com</code> with <code>AWS:SourceArn</code> = this distribution. |

| Topic                        | How it is handled                                                                                                                                                                                                  |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cache invalidation           | Hashed asset names never need invalidating; only <code>/index.html</code> is invalidated per release ( <code>create-invalidation --paths /index.html</code> ).                                                     |
| SPA routing                  | S3 returns 403 for unknown keys (no ListBucket), so <code>custom_error_response</code> maps 403/404 → <code>/index.html</code> with 200.                                                                           |
| API endpoint per environment | Build-time variable ( <code>VITE_API_BASE_URL</code> ) per environment build, or upload a small <code>config.json</code> per environment (no-cache) that the app fetches at start, so one build can be promoted.   |
| Certificate                  | CloudFront only accepts ACM certificates from <code>us-east-1</code> → <code>aws.us_east_1</code> provider alias.                                                                                                  |
| DNS                          | Route 53 A + AAAA <code>alias</code> records to the distribution ( <code>hosted_zone_id</code> from the resource).                                                                                                 |
| CORS                         | The API must allow origin <code>https://app.example.com</code> (application setting).                                                                                                                              |
| Old assets                   | <code>--delete</code> removes old hashed files; browsers holding an old <code>index.html</code> can 404 on lazy chunks. Omit <code>--delete</code> and expire old files with an S3 lifecycle rule if this matters. |

**Caution** Alternative: route `/api/*` through the same CloudFront distribution to the ALB (same origin, no CORS). If you do, the distribution-wide 403/404 → `index.html` rewrite also applies to API responses. Use a CloudFront Function on the default behavior for SPA routing instead, and a no-cache policy for `/api/*`.

### Why React does not run inside ECS here

- Static files need no CPU: CloudFront serves them from edge caches, so the API fleet only handles API requests.
- No container to patch, scale or pay for per hour; S3 + CloudFront are pay-per-use.
- Frontend and API deploy independently.

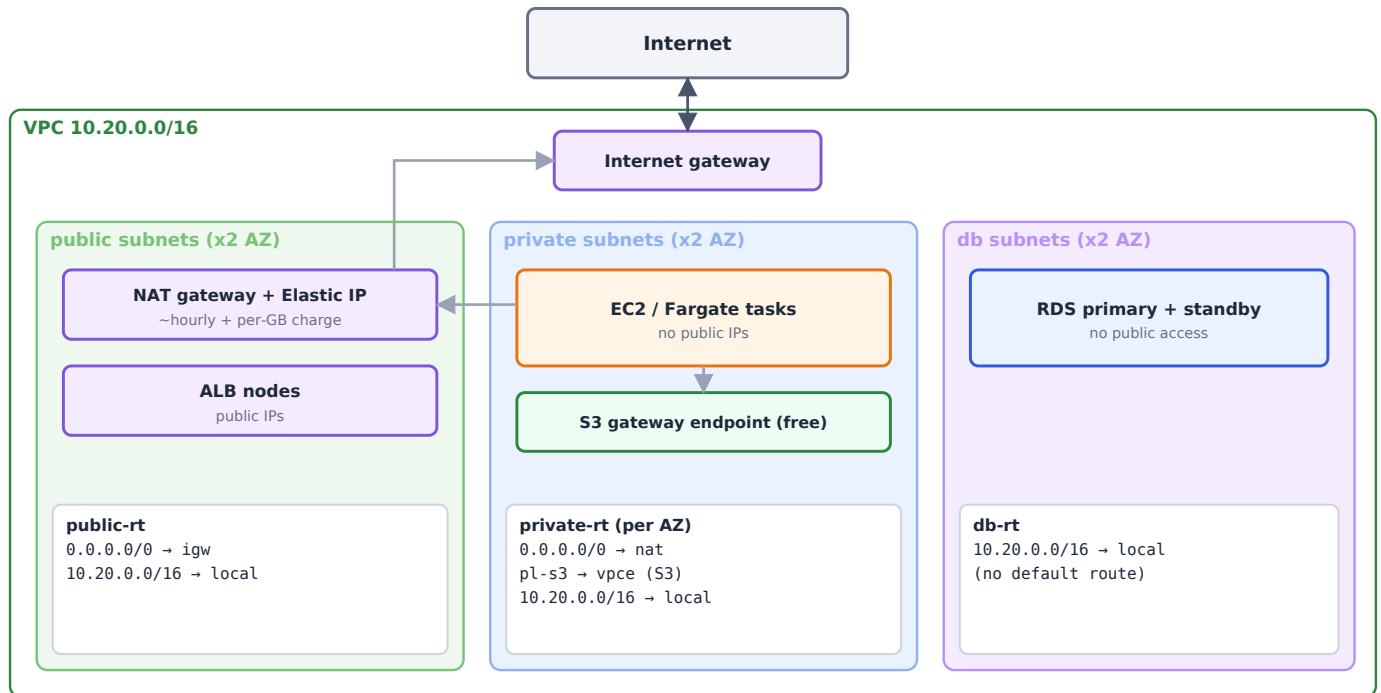
### When a container for the frontend is reasonable

- Server-side rendering (e.g. Next.js SSR) or a Node backend-for-frontend.
- Runtime environment injection that must happen on the server.
- A requirement for one artifact per release or for private, VPC-only hosting behind an internal ALB.

## PART 7

# Networking and security

## 7.1 VPC, subnets and routing

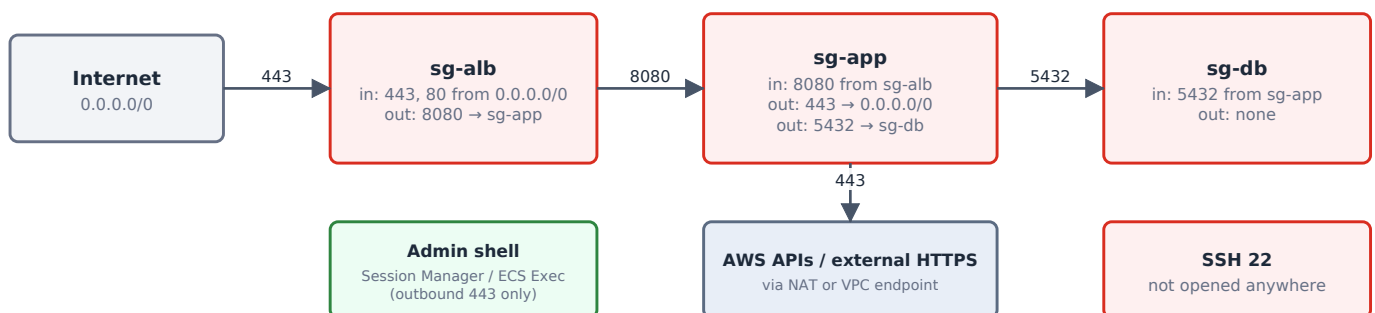


Three subnet tiers per AZ. Only public subnets route to the internet gateway; private subnets go out through NAT; db subnets have no default route at all.

### network.tf per-AZ private routing

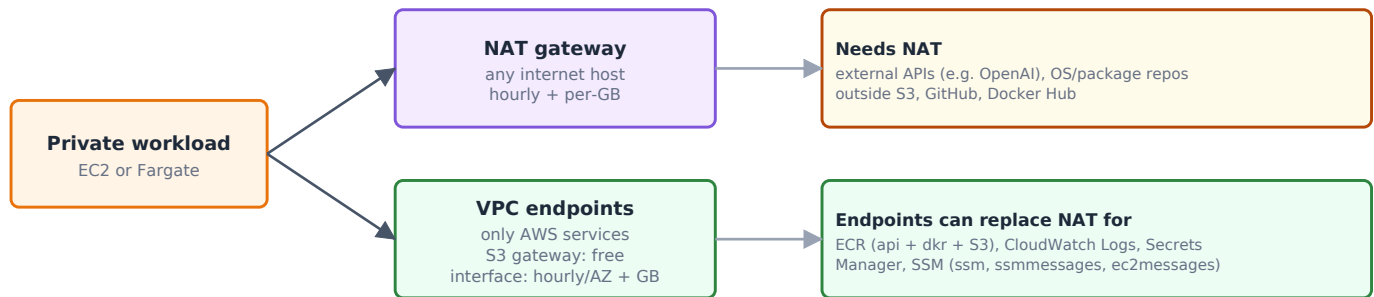
```
# One route table per AZ so each AZ can use its own NAT when single_nat_gateway = false.
resource "aws_route_table" "private" {
  count = var.az_count
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.main[var.single_nat_gateway ? 0 : count.index].id
  }
  tags = { Name = "${local.name}-private-rt-${local.azs[count.index]}" }
}
```

## 7.2 Security groups: permitted traffic paths



Rules reference security groups, not IP ranges, so they stay correct as tasks/instances come and go. Nothing accepts inbound traffic except the ALB.

## 7.3 Outbound access, NAT and VPC endpoints

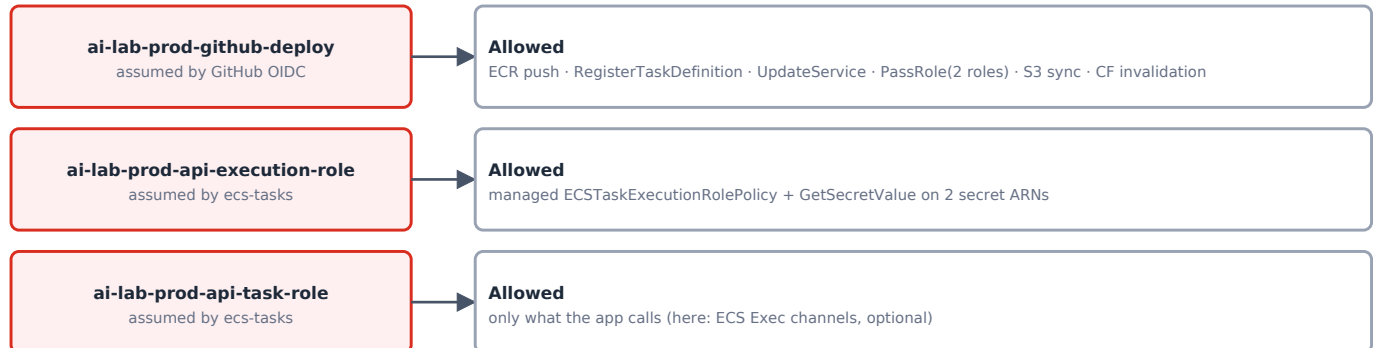


NAT gives private workloads general internet egress. VPC endpoints give private access to specific AWS services. The S3 gateway endpoint is free and already configured.

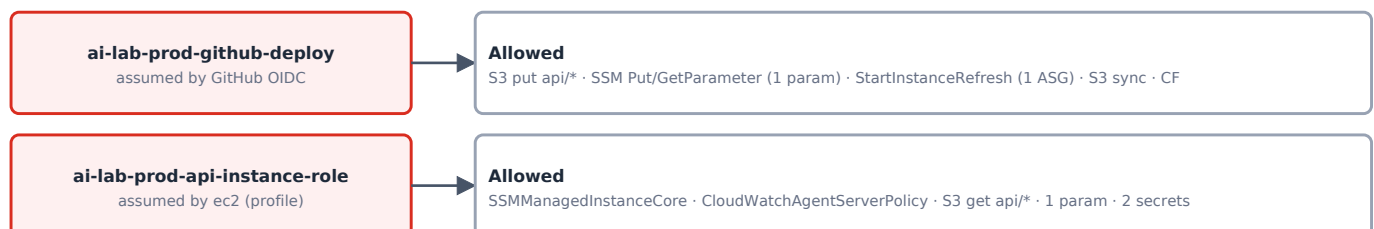
| Component       | Needs outbound to                                                        | Path in this design         | Alternative                                                                             |
|-----------------|--------------------------------------------------------------------------|-----------------------------|-----------------------------------------------------------------------------------------|
| EC2 instances   | OS packages, SSM, CloudWatch, Secrets Manager, S3 (jar), external APIs   | NAT (+ S3 gateway endpoint) | Interface endpoints for ssm, ssmmessages, ec2messages, logs, monitoring, secretsmanager |
| Fargate tasks   | ECR api/dkr + S3 layers, CloudWatch Logs, Secrets Manager, external APIs | NAT (+ S3 gateway endpoint) | Interface endpoints ecr.api, ecr.dkr, logs, secretsmanager                              |
| RDS             | nothing                                                                  | no route                    | —                                                                                       |
| ALB             | only its targets                                                         | VPC local                   | —                                                                                       |
| CloudFront → S3 | AWS-internal                                                             | OAC                         | —                                                                                       |

**Cost** NAT gateways charge per hour and per GB processed (us-east-1 list prices at time of writing ≈ \$0.045/h and \$0.045/GB; check the pricing page). One NAT ≈ \$32+/month before data. Interface endpoints also charge per hour per AZ plus per GB, so 4–6 endpoints in 2 AZs can cost as much as a NAT; they pay off with heavy AWS-API traffic or when the workload must have no internet path. `single_nat_gateway = true` saves money but an AZ outage then also removes egress for the other AZ.

## 7.4 IAM roles and least privilege



ECS roles. Each role trusts exactly one principal and lists exact resource ARNs.



EC2 roles.

**iam.tf (ECS)** CI deploy permissions

```
# ---- GitHub Actions: push image + roll the service -----
data "aws_iam_policy_document" "github_ecs" {
  statement {
    sid      = "EcrLogin"
    actions  = ["ecr:GetAuthorizationToken"]
    resources = ["*"]
  }
}
```

Examples use placeholder IDs (11112223333, example.com, OWNER). Replace before use.

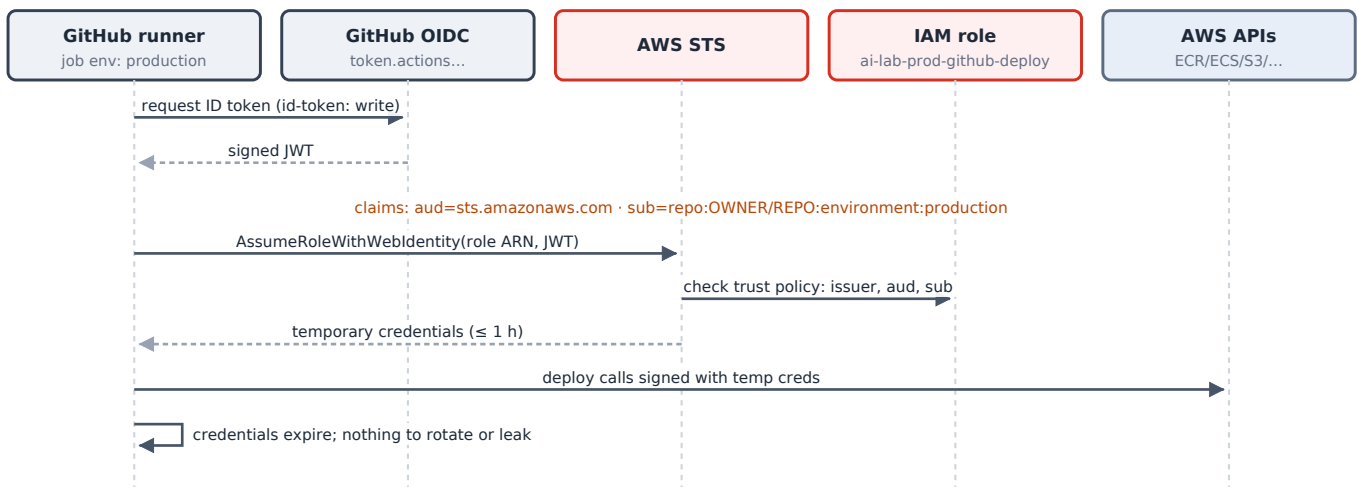
**iam.tf (ECS) CI deploy permissions**

```

actions = ["ecr:GetAuthorizationToken"]
resources = ["*"] # this action does not support resource-level permissions
}
statement {
  sid = "EcrPushPull"
  actions = [
    "ecr:BatchCheckLayerAvailability",
    "ecr:BatchGetImage",
    "ecr:CompleteLayerUpload",
    "ecr:DescribeImages",
    "ecr:InitiateLayerUpload",
    "ecr:PutImage",
    "ecr:UploadLayerPart",
  ]
  resources = [aws_ecr_repository.api.arn]
}
statement {
  sid = "TaskDefinitions"
  actions = ["ecs:DescribeTaskDefinition", "ecs:RegisterTaskDefinition"]
  resources = ["*"] # these actions do not support resource-level permissions
}
statement {
  sid = "RollService"
  actions = ["ecs:UpdateService", "ecs:DescribeServices"]
  resources = [aws_ecs_service.api.id] # the service ARN
}
statement {
  sid = "PassOnlyTheTaskRoles"
  actions = ["iam:PassRole"]
  resources = [aws_iam_role.ecs_execution.arn, aws_iam_role.ecs_task.arn]
  condition {
    test = "StringEquals"
    variable = "iam:PassedToService"
    values = ["ecs-tasks.amazonaws.com"]
  }
}
}
}

```

## 7.5 GitHub Actions OIDC instead of access keys



No long-lived AWS keys exist anywhere. A token from any other repository, branch-only job or environment does not match the **sub** condition and is rejected.

## 7.6 Secrets, HTTPS, logs and alarms

| Area              | Configuration                                                                                                                     | Notes                                                                                            |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Secrets           | RDS-managed master secret ( <code>manage_master_user_password</code> ) + <code>ai-lab/prod/api</code> created by CLI              | Values never in Git, tfvars, state, user-data or task definition. Rotation: see Part 3 caution   |
| HTTPS at the edge | CloudFront <code>TLSv1.2_2021</code> ; ALB <code>ELBSecurityPolicy-TLS13-1-2-2021-06</code> ; HTTP → 301                          | ACM certificates renew automatically while the validation CNAMEs stay in DNS                     |
| Inside the VPC    | ALB → target is HTTP on 8080                                                                                                      | Private network, SG-restricted. Add TLS to targets only if policy requires end-to-end encryption |
| Database          | <code>rds.force_ssl = 1</code> , storage encrypted, not publicly accessible                                                       | Clients must use TLS ( <code>sslmode=require</code> or stricter)                                 |
| Instance metadata | IMDSv2 required, hop limit 1                                                                                                      | Blocks SSRF-style credential theft                                                               |
| Logs              | <code>/ecs/ai-lab-prod-api</code> or <code>/ec2/ai-lab-prod-api</code> , 30-day retention; RDS <code>postgresql</code> log export | Retention controls cost; never log secrets                                                       |
| Alarms            | 5xx, p95 latency, unhealthy hosts, DB CPU, connections, free storage → SNS                                                        | Thresholds are illustrative                                                                      |
| Health endpoint   | ALB checks <code>/actuator/health</code>                                                                                          | Expose only status publicly (application setting); avoid detailed health output on the internet  |

**Security** Never: a publicly accessible RDS instance, `0.0.0.0/0` on port 22 or 5432, passwords in tfvars/user-data/task-definition `environment`, AWS access keys in GitHub secrets or in Git, `*` resources on write actions, or committed Terraform state. Optional hardening: AWS WAF on CloudFront/ALB, GuardDuty, Security Hub, ALB access logs, customer-managed KMS keys, pinning actions and base images to digests.

## PART 8

# Scaling: 2,000 concurrent and 10,000 peak users

No architecture 'supports N users' by itself. Capacity is a measured property of your code, your queries and your data. This part gives the method and an illustrative starting configuration to be replaced by load-test results.

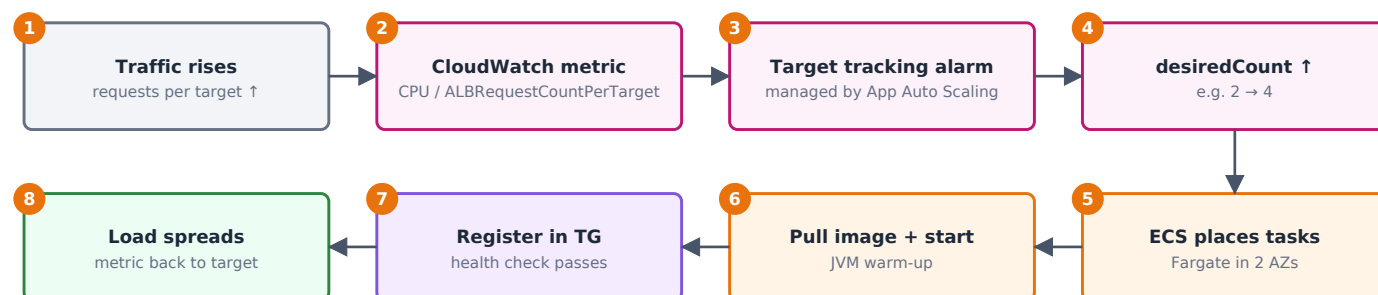
## 8.1 From users to requests

| Step                   | Formula / action                                                                                                               | Illustrative only (assumptions, not measurements)             |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| 1 · Users → RPS        | $RPS \approx \text{active users} \div \text{seconds between requests per user}$                                                | 2,000 users × 1 req / 10 s ≈ 200 RPS; 10,000 peak ≈ 1,000 RPS |
| 2 · In-flight requests | Little's law: $\text{concurrency} = RPS \times \text{avg latency}$                                                             | 1,000 RPS × 0.2 s ≈ 200 requests in flight                    |
| 3 · Per-task capacity  | Load test ONE task/instance: highest RPS with p95 < target, errors < 0.1%, CPU < ~70%                                          | Unknown until measured; call it R                             |
| 4 · Fleet size at peak | $\text{ceil}(\text{peak RPS} \div R) + \text{AZ-loss headroom}$                                                                | If R were 150 → 7 tasks; max 10 leaves headroom               |
| 5 · Minimum            | Survive one AZ loss at normal load: $\text{min} \geq 2$ and $\geq (\text{normal RPS} \div R) \times 2$                         | e.g. min 2-4                                                  |
| 6 · Database           | $\text{max tasks} \times \text{pool size} + \text{admin headroom} \leq \text{max\_connections}$ ; DB CPU/IO from the same test | 10 × 5 = 50 connections                                       |

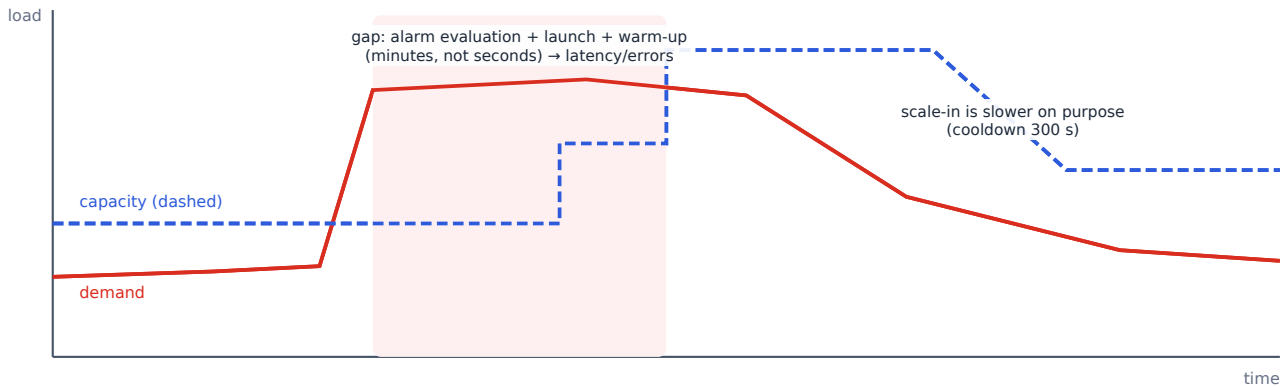
**Caution** Replace the user-behaviour assumption (1 request every 10 s) with real analytics, and run tests against a production-like environment with production-like data volume. Tools: k6, Gatling, JMeter, or the AWS 'Distributed Load Testing on AWS' solution. Ramp gradually, watch RDS as closely as the API, and follow AWS's testing policies.

## 8.2 EC2 Auto Scaling group vs ECS Service Auto Scaling

|                      | EC2 Auto Scaling group                                                                                    | ECS Service Auto Scaling                                                                                  |
|----------------------|-----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| What scales          | Instances (VMs)                                                                                           | Tasks (desiredCount)                                                                                      |
| Min / desired / max  | <code>asg_min_size</code> / <code>asg_desired_capacity</code> / <code>asg_max_size</code>                 | <code>service_min_count</code> / <code>service_desired_count</code> / <code>service_max_count</code>      |
| Policies here        | Target tracking: <code>ASGAverageCPUUtilization</code> 50, <code>ALBRequestCountPerTarget</code> 1000/min | Target tracking: <code>ECSAverageCPUUtilization</code> 50, <code>ALBRequestCountPerTarget</code> 1000/min |
| Health replacement   | <code>health_check_type</code> = ELB                                                                      | Service replaces tasks failing ALB/container checks                                                       |
| AZ balance           | ASG rebalances across <code>vpc_zone_identifier</code>                                                    | Service spreads tasks across subnets/AZs                                                                  |
| Time to add capacity | Boot + user-data + JVM (minutes; measure)                                                                 | Image pull + JVM (often faster; measure)                                                                  |
| Who owns desired     | <code>ignore_changes</code> = [ <code>desired_capacity</code> ]                                           | <code>ignore_changes</code> = [ <code>desired_count</code> ]                                              |



Scale-out chain (ECS). Each arrow adds delay: metric period, alarm evaluation (target tracking alarms need several consecutive datapoints), placement, start-up, health checks.



Mitigations: higher min capacity, lower target utilisation, scheduled scaling before known peaks, faster start-up, CloudFront caching.

Sudden spikes outrun reactive scaling. Keep headroom (min capacity, 50% CPU target) and use scheduled scaling ahead of known peaks.

#### autoscaling.tf ILLUSTRATIVE target

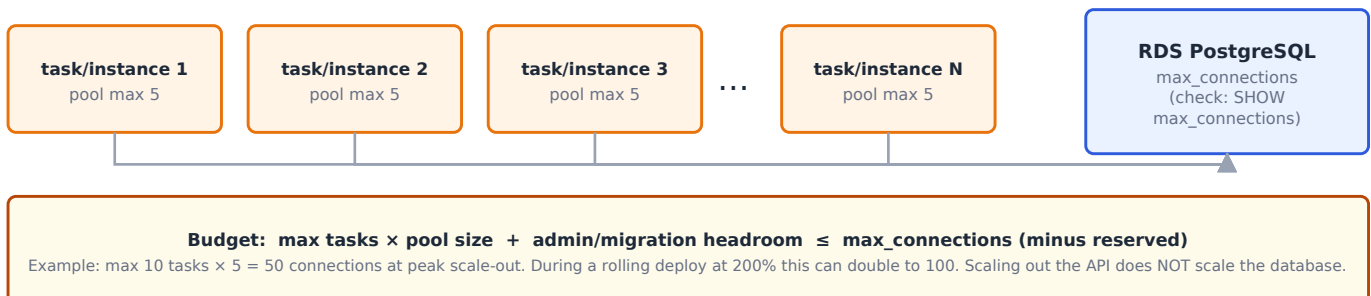
```
resource "aws_appautoscaling_policy" "api_requests" {
  name           = "${local.name}-api-requests"
  policy_type    = "TargetTrackingScaling"
  service_namespace = aws_appautoscaling_target.api.service_namespace
  resource_id    = aws_appautoscaling_target.api.resource_id
  scalable_dimension = aws_appautoscaling_target.api.scalable_dimension

  target_tracking_scaling_policy_configuration {
    predefined_metric_specification {
      predefined_metric_type = "ALBRequestCountPerTarget"
      resource_label        = "${aws_lb.main.arn_suffix}/${aws_lb_target_group.api.arn_suffix}"
    }
    target_value      = 1000 # requests per target per minute - REPLACE with a load-tested value
    scale_out_cooldown = 60
    scale_in_cooldown = 300
  }
}
```

#### scheduled scaling

```
# Scheduled pre-scaling before a known daily peak (ILLUSTRATIVE values; times in UTC)
aws application-autoscaling put-scheduled-action --service-namespace ecs \
--scalable-dimension ecs:service:DesiredCount --resource-id service/ai-lab-prod/ai-lab-prod-api \
--scheduled-action-name pre-peak --schedule "cron(45 8 * * ? *)" \
--scalable-target-action MinCapacity=6,MaxCapacity=10
aws autoscaling put-scheduled-update-group-action --auto-scaling-group-name ai-lab-prod-api-asg \
--scheduled-action-name pre-peak --recurrence "45 8 * * *" --min-size 6
```

## 8.3 Database capacity and connections



Scaling the API multiplies database connections. Check `SHOW max_connections`; and alarm on `DatabaseConnections (db_connections_alarm_threshold)`.

- Scale RDS vertically (instance class) based on the load test; storage autoscaling is on (`max_allocated_storage`).
- Optional: RDS Proxy pools connections across many tasks (extra cost). Optional: read replicas for read-heavy endpoints (application must route reads).
- A Multi-AZ standby is NOT a read replica; it only takes over on failover.

## 8.4 Metrics to watch and cost trade-offs

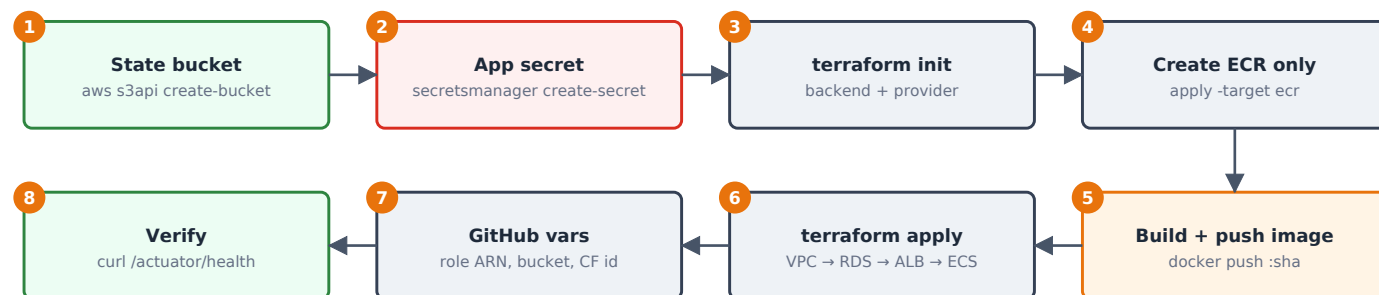
| Metric                                                                                                                         | Source                                                                         | Use                                 |
|--------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|-------------------------------------|
| <code>RequestCountPerTarget</code> , <code>TargetResponseTime</code> p95/p99, <code>HTTPCode_Target_5XX_Count</code>           | ALB                                                                            | SLO, scaling target, alarm rollback |
| <code>CPUUtilization</code> , <code>MemoryUtilization</code>                                                                   | ECS service / Container Insights; EC2 + CW agent <code>mem_used_percent</code> | Right-size task/instance            |
| <code>DatabaseConnections</code> , <code>CPUUtilization</code> , <code>FreeableMemory</code> , <code>ReadIOPS/WriteIOPS</code> | RDS                                                                            | DB is usually the first bottleneck  |
| <code>CacheHitRate</code>                                                                                                      | CloudFront                                                                     | Static traffic kept off the origin  |

| Lever                          | Performance                                                  | Cost                                                                                          |
|--------------------------------|--------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Higher minimum capacity        | Absorbs spikes, survives AZ loss                             | Pay for idle capacity                                                                         |
| Lower CPU target (e.g. 40-50%) | More headroom                                                | More tasks/instances                                                                          |
| Fargate vs EC2                 | Similar per vCPU; Fargate starts faster, no bin-packing work | Fargate per vCPU-hour/GB-hour; EC2 cheaper when well packed; Compute Savings Plans cover both |
| Graviton (ARM64)               | Often better price/performance                               | Requires arm64 image/AMI and testing                                                          |
| Fargate Spot / EC2 Spot        | Can be interrupted                                           | Large discount; only for interruption-tolerant extra capacity                                 |
| Multi-AZ RDS                   | Failover in ~1-2 min                                         | ~2× DB instance cost                                                                          |

## PART 9

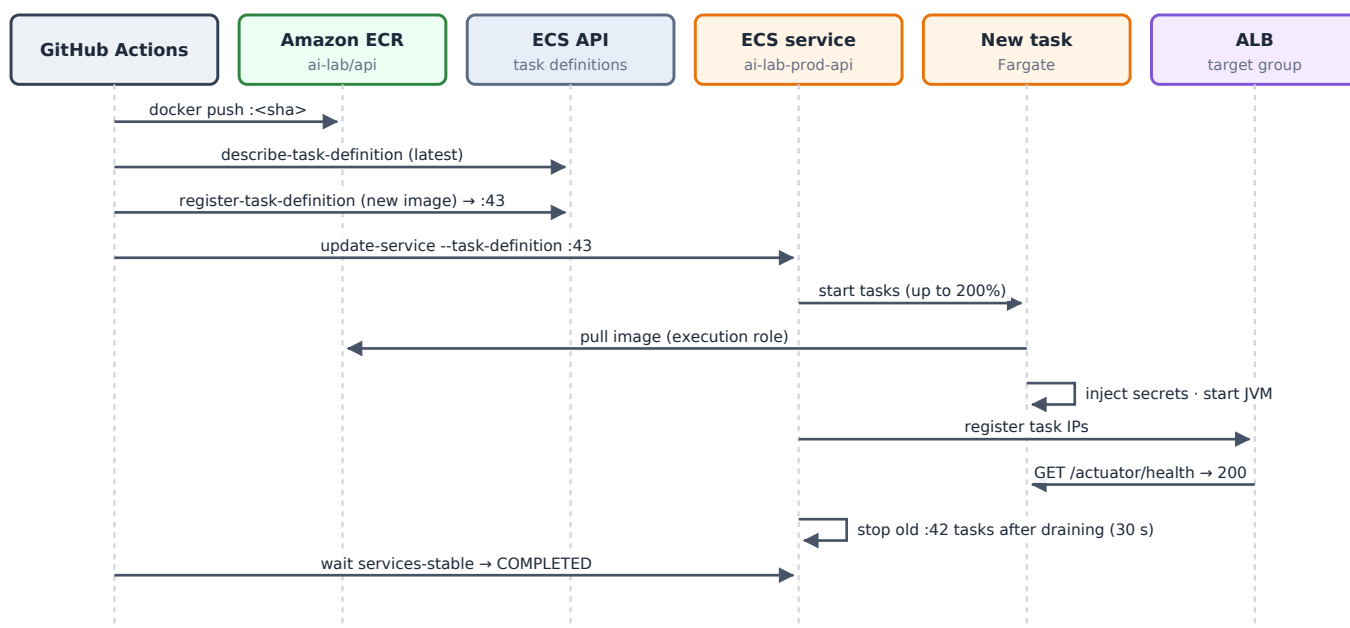
# Deployment and operational diagrams

## 9.1 First-time deployment



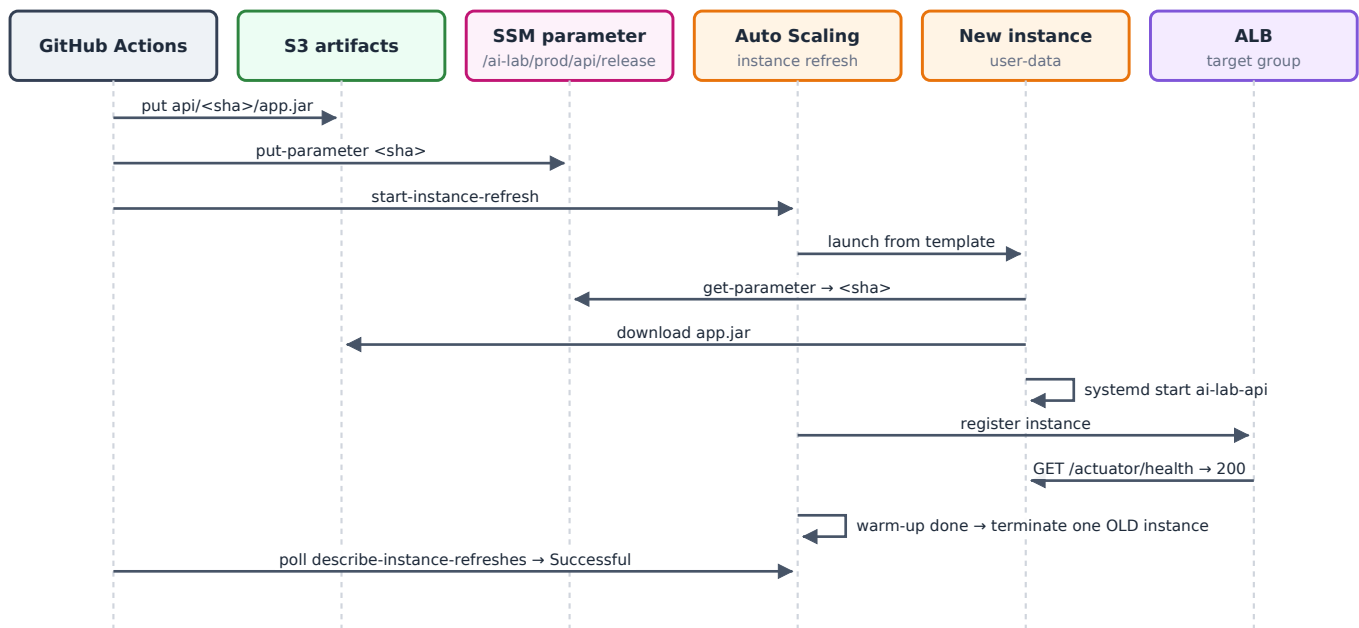
Trigger: an operator, once per environment. Performed by: you (CLI + Terraform). EC2 variant: Part 3.

## 9.2 Normal application update (ECS)



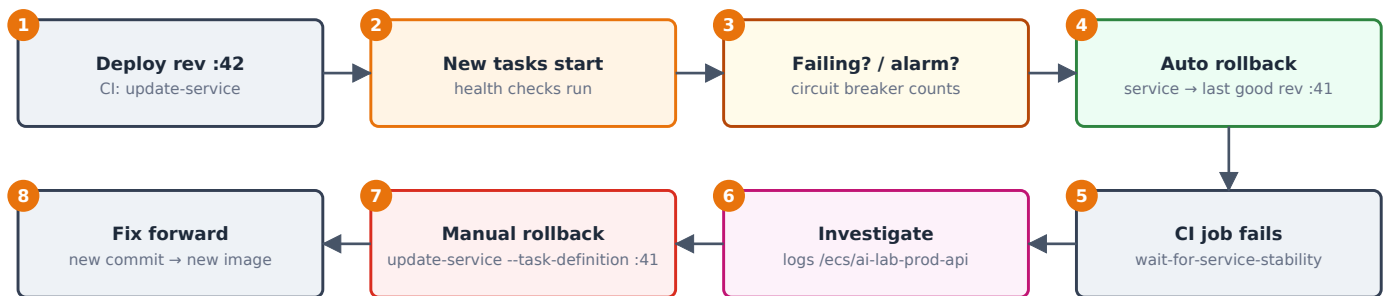
Trigger: push to **main**. Performed by: GitHub Actions (OIDC role), then the ECS service scheduler.

## 9.2b Normal application update (EC2)



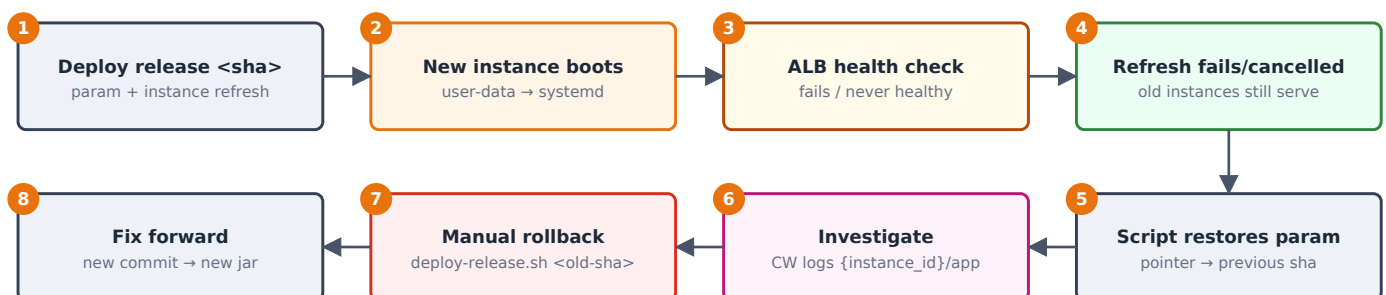
Trigger: push to **main**. Performed by: GitHub Actions + **deploy-release.sh**, then the ASG instance refresh.

## 9.3 Failed deployment and rollback (ECS)



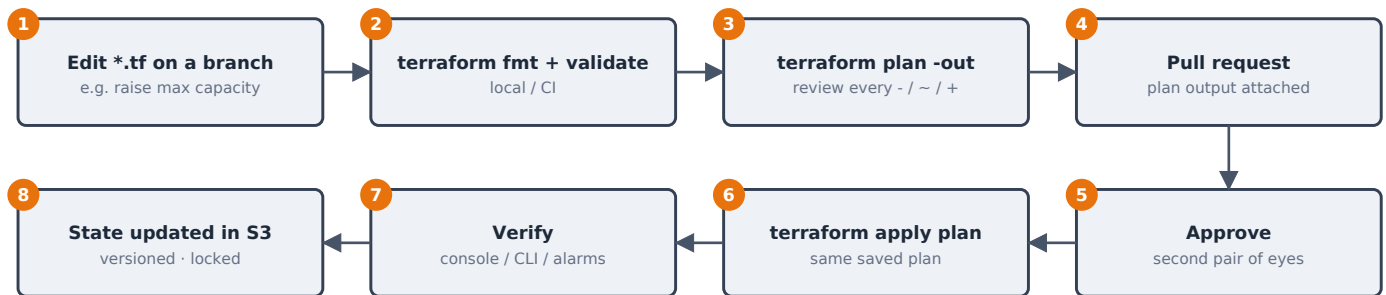
Trigger: new tasks failing or the 5xx alarm. Performed by: ECS circuit breaker / alarm rollback automatically; manual rollback by an operator.

## 9.3b Failed deployment and rollback (EC2)



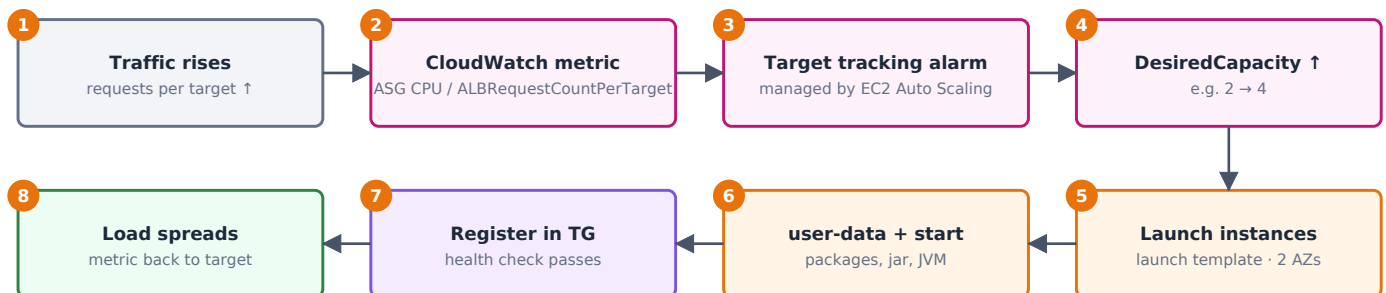
Trigger: refresh failing health checks. Performed by: ASG (stops), the script (restores pointer), operator (redeploy old SHA).

## 9.4 Infrastructure change through Terraform



Trigger: a reviewed pull request. Performed by: an engineer or a CI job with a separate, more privileged Terraform role. Never apply an unreviewed plan.

## 9.5 Application scaling (EC2)



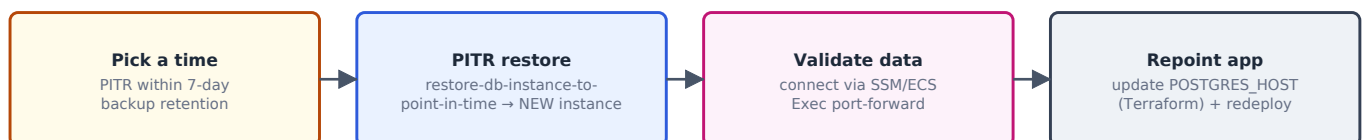
Trigger: CloudWatch target-tracking alarms. Performed by: EC2 Auto Scaling (ECS variant in Part 8).

## 9.6 Database failure and recovery

### A · Multi-AZ failover (automatic)



### B · Restore (data corruption, bad migration, deleted rows) — you initiate

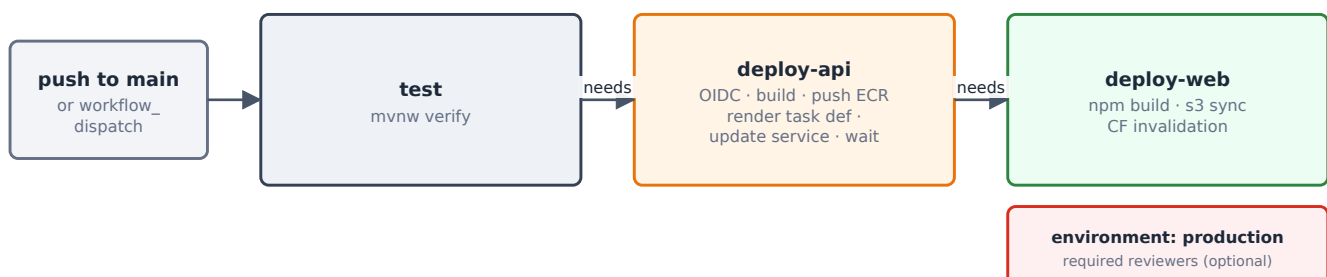


Failover typically completes in about a minute or two (AWS documents 60–120 s for many cases); measure it with a test failover:  
`aws rds reboot-db-instance --db-instance-identifier ai-lab-prod-db --force-failover`

A: triggered by RDS health monitoring, performed by RDS. B: triggered and performed by you; Terraform must be updated to the restored instance.

## 9.7 GitHub Actions CI/CD (ECS)

concurrency group deploy-prod: one production deployment at a time, never cancelled midway.  
 Failure anywhere stops later jobs: the UI is only published after the API it calls is healthy.



Trigger: push to **main** or manual dispatch. Each job assumes the deploy role via OIDC.

## 9.7b GitHub Actions CI/CD (EC2)

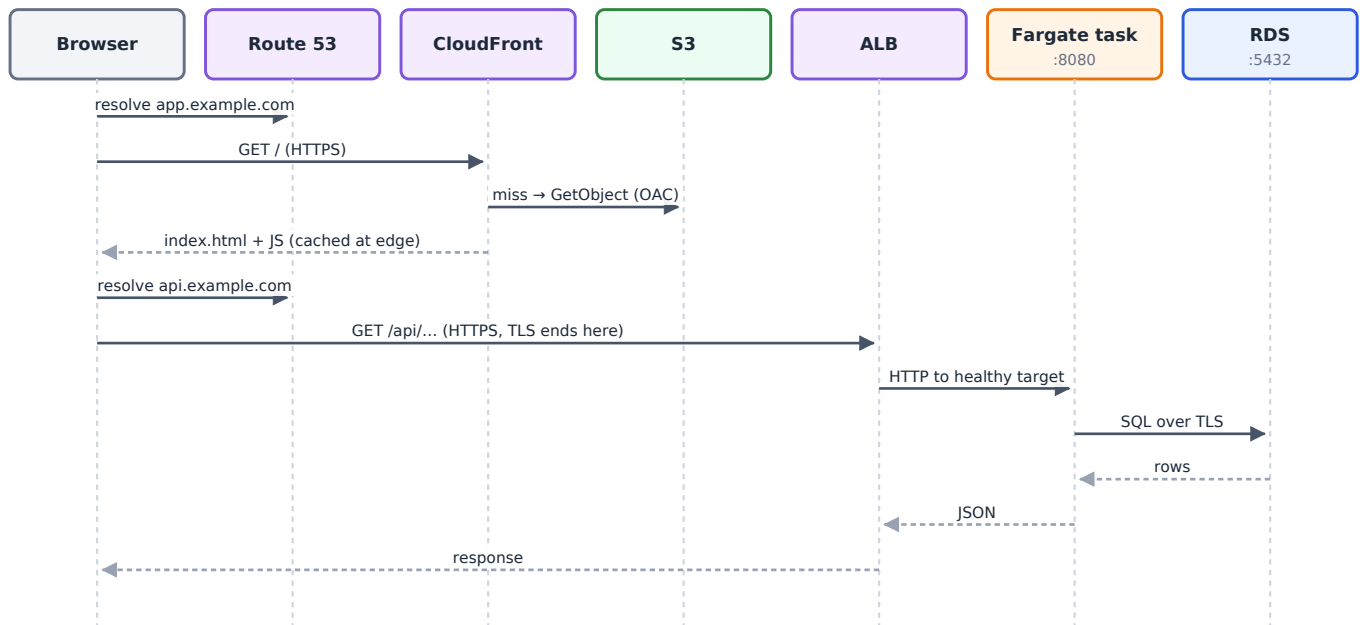
concurrency group deploy-prod: one production deployment at a time, never cancelled midway.

Failure anywhere stops later jobs: the UI is only published after the API it calls is healthy.



Manual dispatch with `release=<sha>` skips the build and performs a rollback.

## 9.8 Request flow: browser to database



Static requests end at CloudFront (or S3 on a cache miss). API requests: TLS ends at the ALB, which forwards to a healthy target; the API talks TLS to RDS.

## PART 10

# Final comparison, recommendation and checklist

| Dimension                   | A · EC2                                                        | B · ECS Fargate                                                     |
|-----------------------------|----------------------------------------------------------------|---------------------------------------------------------------------|
| Infrastructure management   | VMs, AMIs, launch template, ASG                                | Cluster, task definitions, service; no hosts                        |
| Configuration files         | Terraform + user-data + systemd unit + deploy script           | Terraform + Dockerfile (+ .dockerignore)                            |
| Deployment complexity       | Jar + release pointer + instance refresh                       | Image + new task-def revision + service update                      |
| Autoscaling                 | EC2 Auto Scaling target tracking                               | Application Auto Scaling on desiredCount                            |
| Security responsibilities   | Plus OS hardening, patching, agents, IMDS                      | Image contents and base-image updates                               |
| OS maintenance              | Yours (new AMI → refresh)                                      | AWS (Fargate platform)                                              |
| Container requirement       | None                                                           | Container image required                                            |
| Cost                        | Instance-hours; cheapest when well utilised; RIs/Savings Plans | vCPU/GB-hours per task; no idle host capacity; Savings Plans        |
| Observability               | CW agent for logs + memory/disk                                | <b>awslogs</b> + Container Insights built in                        |
| Rollback                    | Re-point param + refresh (minutes)                             | Automatic (circuit breaker/alarm) or update-service to old revision |
| Operational effort          | Higher                                                         | Lower                                                               |
| First production deployment | Good if you need host control or cannot containerise           | Recommended default for this stack                                  |

## 10.1 Recommendation

For a React + single API + PostgreSQL application whose backend already runs as a self-contained process, **start with ECS on Fargate**. It removes OS patching and host capacity work, gives automatic deployment rollback, and keeps the same VPC/ALB/RDS/CloudFront foundation as the EC2 design. Choose EC2 when you need host-level agents, special instance types, very steady high utilisation where packed instances are cheaper, or when the team cannot yet operate containers. Because the shared files are identical, moving from A to B later is a compute-layer swap.

## 10.2 Files to create first (ECS path)

| #  | Item                                                                   | Created by | Done when                                                                  |
|----|------------------------------------------------------------------------|------------|----------------------------------------------------------------------------|
| 1  | State bucket <b>ai-lab-tfstate-&lt;account&gt;</b>                     | CLI (once) | Versioned, public access blocked                                           |
| 2  | App secret <b>ai-lab/prod/api</b>                                      | CLI (once) | JSON contains every key the task definition references                     |
| 3  | <b>versions.tf, providers.tf, variables.tf, terraform.tfvars</b>       | You        | <b>terraform init + validate</b> pass                                      |
| 4  | <b>network.tf, security-groups.tf</b>                                  | You        | Plan shows 3 subnet tiers × 2 AZs, no SSH rules                            |
| 5  | <b>rds.tf, dns-acm.tf, load-balancer.tf</b>                            | You        | Cert ISSUED; ALB answers 301 on :80                                        |
| 6  | <b>Dockerfile, .dockerignore, ecr.tf</b>                               | You        | Image pushed with a SHA tag                                                |
| 7  | <b>iam.tf, ecs-cluster.tf, ecs-task-definition.tf, ecs-service.tf</b>  | You        | Service COMPLETED, targets healthy                                         |
| 8  | <b>autoscaling.tf, monitoring.tf</b>                                   | You        | Policies listed; alarms OK; SNS confirmed                                  |
| 9  | <b>frontend.tf, github-oidc.tf, outputs.tf</b>                         | You        | <b>https://app.example.com</b> serves the SPA                              |
| 10 | <b>.github/workflows/deploy-ecs.yml</b> + GitHub environment/variables | You        | A push deploys; a broken image rolls back                                  |
| 11 | <b>.gitignore</b> additions                                            | You        | <b>*.tfstate*, .terraform/, terraform.tfvars, tfplan, *.tfplan</b> ignored |
| 12 | Load test + tuned capacity values                                      | You        | min/max/targets based on measurements                                      |

## 10.3 State management

- One state per stack and environment (**prod/ecs/terraform.tfstate**). Never share state between stacks.
- S3 versioning lets you recover a corrupted state; **use\_lockfile** prevents two concurrent applies.
- State contains resource attributes in plain text. Here no password is in state (RDS-managed secret, CLI-created app secret), but endpoints, ARNs and any future secret-version resources would be. Restrict bucket access to the people/roles that run Terraform.
- Saved plans (**tfplan**) can contain sensitive values too: keep them out of Git and artifacts.

## 10.4 Cleanup and ongoing charges

### cleanup

```
# 1. remove deletion protection (edit: enable_deletion_protection=false on aws_lb.main, deletion_protection=false
#    on aws_db_instance.main, optionally skip_final_snapshot=true), then:
terraform apply
# 2. empty buckets Terraform cannot delete while they contain objects
aws s3 rm s3://ai-lab-prod-frontend-111122223333 --recursive
aws s3api delete-objects --bucket ai-lab-prod-artifacts-111122223333 --delete "$(aws s3api list-object-versions \
--bucket ai-lab-prod-artifacts-111122223333 \
--query '{Objects: Versions[].{Key:Key,VersionId:VersionId}}' --output json)"
# repeat with DeleteMarkers[] instead of Versions[]; max 1000 keys per call (or use the console's 'Empty bucket')
aws ecr batch-delete-image --repository-name ai-lab/api \
--image-ids "$(aws ecr list-images --repository-name ai-lab/api --query imageIds --output json)"
# 3. destroy, then remove what lives outside Terraform
terraform destroy
aws secretsmanager delete-secret --secret-id ai-lab/prod/api --recovery-window-in-days 7
aws rds describe-db-snapshots --db-instance-identifier ai-lab-prod-db # final snapshot is billed until deleted
```

| Keeps charging while it exists                                                        | Notes                                                              |
|---------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| NAT gateway(s), public IPv4 addresses (NAT EIP, ALB)                                  | Hourly, even with zero traffic                                     |
| ALB                                                                                   | Hourly + LCU usage                                                 |
| RDS instance + storage + backups/snapshots                                            | Multi-AZ $\approx 2 \times$ instance; final snapshot after destroy |
| Fargate tasks / EC2 instances + EBS                                                   | Per second/hour while running                                      |
| Secrets Manager secrets, KMS keys, CloudWatch logs/metrics/alarms, Container Insights | Small but recurring                                                |
| CloudFront, S3, ECR storage, data transfer                                            | Usage-based                                                        |

**Cost** Set an AWS Budget with an e-mail alert before the first apply, and review Cost Explorer after the first week. Prices vary by region and change over time; use the AWS Pricing Calculator for your numbers.

## 10.5 Final deployment checklist

| <input type="checkbox"/> | Check                                                                                                                     |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> | State bucket versioned and private; <code>.gitignore</code> covers state, tfvars and plans                                |
| <input type="checkbox"/> | No secret values in Git, tfvars, user-data, task-definition <code>environment</code> or Terraform state                   |
| <input type="checkbox"/> | RDS: private subnets, <code>publicly_accessible = false</code> , encrypted, Multi-AZ, backups 7 days, deletion protection |
| <input type="checkbox"/> | Security groups: only 443/80 public on the ALB; 8080 from ALB; 5432 from app; no port 22                                  |
| <input type="checkbox"/> | ACM certificates ISSUED (regional for ALB, us-east-1 for CloudFront); HTTP redirects to HTTPS                             |
| <input type="checkbox"/> | S3 frontend bucket private, accessed only via OAC; SPA deep links return 200                                              |
| <input type="checkbox"/> | Targets healthy in 2 AZs; killing one task/instance self-heals                                                            |
| <input type="checkbox"/> | Rolling deploy tested; a deliberately broken release rolls back (ECS) or stops safely (EC2)                               |
| <input type="checkbox"/> | Alarms OK and SNS subscription confirmed; logs visible with sensible retention                                            |
| <input type="checkbox"/> | GitHub OIDC role restricted to repo + <code>production</code> environment; no AWS keys in GitHub                          |
| <input type="checkbox"/> | Load test done; capacity numbers replaced; DB connection budget checked                                                   |
| <input type="checkbox"/> | Restore drill done (PITR to a new instance) and a test failover performed                                                 |
| <input type="checkbox"/> | AWS Budget alert configured; cleanup procedure understood                                                                 |